

Process MeNtOR 3.0

Uni-SEP

Content adapted from the deliverables prepared by Dr. Kostas Kontogiannis

EV Marketplace:

A Cloud-Native Electric Vehicle

E-Commerce Platform

Design Document

EECS 4413 - Deliverable 3

Team Members:

Kiana Misaghi

Minh Anh Nguyen

Tazwar Sikder

Matin Pakfetrat

Document Change Control

Version	Date	Authors	Summary of Changes
1	28th May 2026	Minh Anh Nguyen, Kiana Misaghi	Create document, divided section with assigned name
2	30th May 2026	Tazwar Sikder, Matin Pakfetrat, Kiana	Added some draft on assigned section
3	5th June 2026	Minh Anh Nguyen	Added use case diagram, added architecture decisions
4	6th June 2026	Tazwar Sikder, Matin Pakfetrat, Kiana Misaghi, Minh Anh Nguyen	Modify the drafts, adjusting information.
5	7th June 2026	Tazwar Sikder, Kiana Misaghi	Added sequence diagrams for sign in / sign out and add to cart.
6	10th June 2026	Tazwar Sikder, Kiana Misaghi	Added sequence diagram for schedule test drive.
7	12th June 2026	Tazwar Sikder, Matin Pakfetrat, Kiana, Minh Anh Nguyen	Modify and review documents last time, finalize and submit. Added missing distinct Notification service.
8	2nd July 2026	Minh Anh Nguyen	Modify the information and add sections for Deliverable 2
9	13th July 2026	Matin Pakfetrat, Tazwar Sikder, Minh Anh Nguyen	Completed Section 8 detailed design (refined component diagram, deployment diagram, module refinements for Catalog, Cart/Order/Payment, and Test Drive/Analytics/Notification), Section 9 activities plan, Section 10 test coverage, and Section 11 server-side implementation description.
10	28th July 2026	Tazwar Sikder	Start adding sections for Deliverable 3
11	30th July 2026	Minh Anh Nguyen, Matin Pakfetrat, Tazwar Sikder	Add diagrams and web ui images, including descriptions
12	2nd August 2026	Minh Anh Nguyen, Matin Pakfetrat, Tazwar Sikder	Describe Client-Side implementation, chatbot implementation, Test-drive implementation, Web Design, Final Design
13	4th August 2026	Minh Anh Nguyen, Matin Pakfetrat, Tazwar Sikder	Final proofread, adding missing elements

Document Sign-Off

Name (Position)	Signature	Date
Tazwar Sikder (Sequence diagrams, component diagrams, client-side implementation, chat-bot implementation, reflection, Final Design)	TAZWAR SIKDER	August 4th, 2026
Matin Pakfetrat (Architecture, Test Driven Development, Client-side implementation, Test-drive implementation, Activity plans)	MATIN PAKFETRAT	August 4th, 2026
Minh Anh Nguyen (Major Design Decisions, Use Case Diagram, Test Driven Development, Web Design, Client-side implementation, Final Design)	MINH ANH NGUYEN	August 4th, 2026
Kiana Misaghi (Introduction, Activities Plan, Test Driven Development)	KIANA MISAGHI	August 4th, 2026

Contents

1 Introduction	5
1.1 Purpose	6
1.2 Overview	6
1.3 Resources - References	6
2 Major Design Decisions	7
3 Use case Diagram	10
4 Sequence Diagrams	11
4.1 Use Case 1 — Sign in / Sign out:	11
4.2 Use Case 2 — Add vehicle to shopping cart:	11
4.3 Use Case 3 — Schedule a test drive:	12
5 Architecture	13
5.1 Package Diagram	13
5.2 Component Diagram	14
5.3 Module Descriptions	14
5.4 Interface Descriptions	16
6 Activities Plan	18
6.1 Project Backlog and Sprint Backlog	18
6.2 GANTT Chart	21
6.3 Group Meeting Logs	23
7 Test Driven Development	25
8 Detailed Design	29
8.1 Refined Component Design	30
Presentation Layer	30
8.2 Design Pattern Rationale	32
8.3 Deployment Diagram and Cloud-Native Architecture	32
8.4 Quality Attributes Supported	34
8.5 Shopping Cart, Checkout, and Payment — Module Refinement	35
Design pattern rationale	35
Quality attributes supported by this slice	35
8.6 Test Drive, Analytics, and Notification - Module Refinement	35
9 Activities Plan	36
10 Test Coverage	37
11 Server-Side Implementation Description	42
Technologies used	42
REST principles followed	42
Fake payment logic	42
12 Final Design	43
13 Web Design	46
14 Activities Plan for Deliverable 3	53
15 Client-Side Implementation Description	54
15.1 Cart/Checkout/Payment	54

Technologies used	54
Component / page breakdown	55
REST principles followed (client side)	55
15.2 Test Drive Booking	55
Technologies used	55
16 Chatbot Implementation	56
17 Demo Table	57
18 Reflection	58

1 Introduction

1.1 Purpose

The purpose of this project was to create an online electric vehicle (EV) store for customers to find, compare, customize, and buy EVs using a web-based application. This system supports customer functions including registering, logging in, viewing EV catalogs, comparing EVs by price or specifications, calculating loans, managing their cart, completing checkout processes, writing reviews and chatting with our customer service team. Additionally, it supports administrative functions including creating sales reports and analyzing how much the application is being used.

1.2 Overview

The system addresses the issue of developing a digital sales environment for an auto manufacturer who focuses on electric cars (EVs). Customers will be able to view all available electric vehicles; compare them based on various features; determine their estimated monthly payment; and finalize their purchase via the Internet. Administrators will have access to information about sales and usage. The system will utilize a multi-tier architectural design where there is a clear distinction between the front end, back end, Web API's, and databases.

1.3 Resources - References

- [1] GeeksforGeeks, "Singleton Design Pattern," [Online]. Available: <https://www.geeksforgeeks.org/system-design/singleton-design-pattern/>
- [2] GeeksforGeeks, "Repository Design Pattern," [Online]. Available: <https://www.geeksforgeeks.org/system-design/repository-design-pattern/>
- [3] GeeksforGeeks, "Strategy Pattern – Set 1," [Online]. Available: <https://www.geeksforgeeks.org/system-design/strategy-pattern-set-1/>
- [4] C. Richardson, "Pattern: Microservice Architecture," microservices.io. [Online]. Available: <https://microservices.io/patterns/microservices.html>
- [5] C. Richardson, "Pattern: API Gateway," microservices.io. [Online]. Available: <https://microservices.io/patterns/apigateway.html>
- [6] VMware Tanzu, "Spring Data JPA – Reference Documentation," Spring Docs. [Online]. Available: <https://docs.spring.io/spring-data/jpa/reference/jpa.html>
- [7] Amazon Web Services, "PostgreSQL on Amazon RDS," AWS Documentation. [Online]. Available: https://docs.aws.amazon.com/AmazonRDS/latest/UserGuide/CHAP_PostgreSQL.html
- [8] GeeksforGeeks, "MVC Design Pattern," [Online]. Available: <https://www.geeksforgeeks.org/system-design/mvc-design-pattern/>
- [9] GeeksforGeeks, "Component-Based Diagram," [Online]. Available: <https://www.geeksforgeeks.org/software-engineering/component-based-diagram/>
- [10] GeeksforGeeks, "Package Diagram – Introduction, Elements, Use Cases and Benefits," [Online]. Available: <https://www.geeksforgeeks.org/system-design/package-diagram-introduction-elements-use-cases-and-benefits/>
- [11] GeeksforGeeks, "Unified Modeling Language (UML) | Sequence Diagrams," [Online]. Available:

<https://www.geeksforgeeks.org/system-design/unified-modeling-language-uml-sequence-diagrams/>

2 Major Design Decisions

Architectural patterns:

Presentation layer: React frontend

Business Layer: Springboot backend

Data layer: Neon-managed PostgreSQL

REST APIs are used for communication between the front-end and back-end; the front-end communicates with a single Spring Boot application directly, with no separate API gateway. The user interface and server-side logic are separately developed, maintained and tested. Note: at this stage of the design (Deliverable 1), the architecture was planned around a microservice architecture, with the intention that each service would be independently deployable to increase scalability, maintainability, and extensibility. As implemented and deployed (see Sections 8.3 and 12), the system was instead built as a single modular-monolith Spring Boot application: each module below remains a cohesive, loosely-coupled package with its own controller/service/repository, but all modules are deployed together as one process rather than as independently deployed microservices.

Service Decomposition:

Catalog Service:

- Vehicle listings
- Vehicle details
- Vehicle filtering
- Vehicle sorting
- Vehicle comparison
- Hot deals

Ordering Service:

- Checkout processing
- Order creation
- Payment authorization
- Order history

Identity Management Service:

- User registration
- Login and logout
- Authentication and authorization

Shopping Cart Service:

- Adding vehicles to cart
- Updating cart contents
- Removing vehicles from cart
- Calculating totals

Analytics Service:

- Sales reports

- Website usage reports
- Visitor tracking

Chatbot Service:

- Answering frequently asked questions
- Assisting users in locating the vehicles
- Guiding users to customer support pages

Test Drive Service:

- Booking test drive appointments
- Checking slot availability
- Managing booking records
- Triggering confirmation notification

Notification Service:

- Sending order confirmation notifications
- Sending test drive booking confirmations and reminders
- Sending account registration and password reset notifications
- Delivering promotional offers and hot deal alerts
- Managing customer notification preferences
- Tracking notification delivery status

Design patterns:

Strategy pattern: Used to support multiple vehicle shorting and filtering methods, such as sorting by mileage or price in ascending and descending order. The system is easier to extend and add without modifying the existing logic.

Repository pattern: Used to separate database access from business logic. Improves maintainability, testability, and flexibility by allowing services to be easily modified, adapted, and removed separately with the data storage.

Singleton pattern: Ensure that each service only has one instance, which allows global access across the entire application, resource usage is reduced and consistent behavior is maintained throughout the system.

Model-View-Controller (MVC): The MVC would allow the Model, View and Controllers to interact with each other and have minimal impact on each other when created, update, and remove. Controllers use REST APIs gateway to process the client request to the business logic, it mediates between “model” and “views”. The views are the front-end components such as Registration View, Administrator Analytic Page View, Catalog View, Shopping Cart View, Check Out View.

Modularized criteria:

The system is organized using modular package boundaries, in which each module is responsible for a specific business capability. As implemented (see Section 8.3), these modules run as a single deployed Spring Boot application rather than as independently deployed microservices. The goal for modularization is to achieve *high cohesion* and *low coupling*.

High cohesion: Group the related function within the same service.

For example:

- Catalog Service manages vehicle listings, vehicle details, sorting, filtering, comparisons, and hot deals.

- Shopping Cart Service manages shopping cart operations such as adding, removing and updating vehicles.
- Ordering Service handles checkout, payment processing and order management.
- Identity Service manages user registration, authentication, and authorization.
- Analytics Service generates sales and website usage reports.

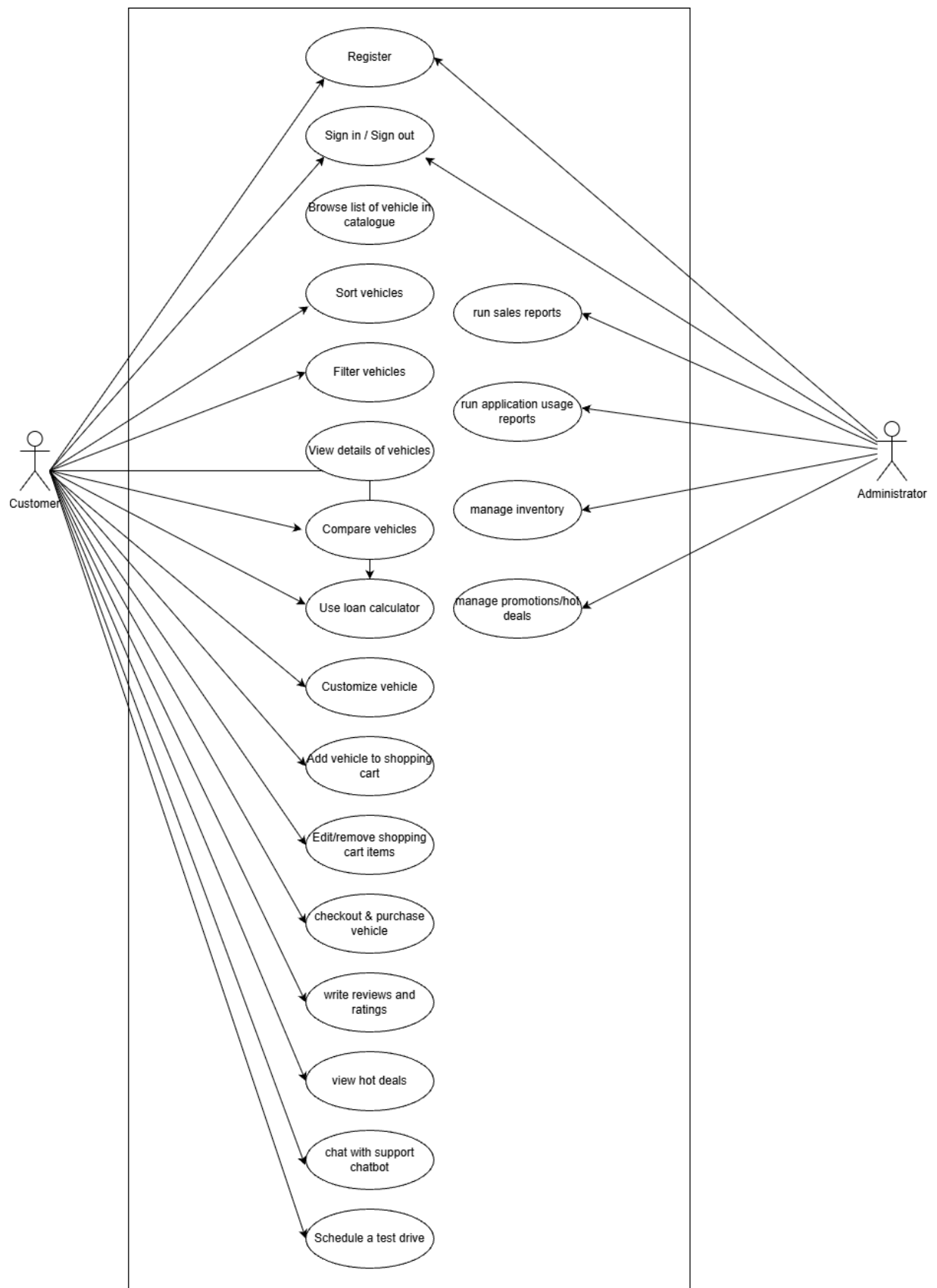
Low coupling: Minimize dependencies between modules. Each module communicates with other modules through well-defined service interfaces (Java method calls within the single deployed Spring Boot application), and does not directly access another module's internal implementation or database tables.

For example:

- Ordering Service can communicate with the Identity Service to verify user information without needing to know how authentication is implemented internally.
- Analytics Service does not directly depend on the Ordering Service's logic, it gets the information from service interactions.

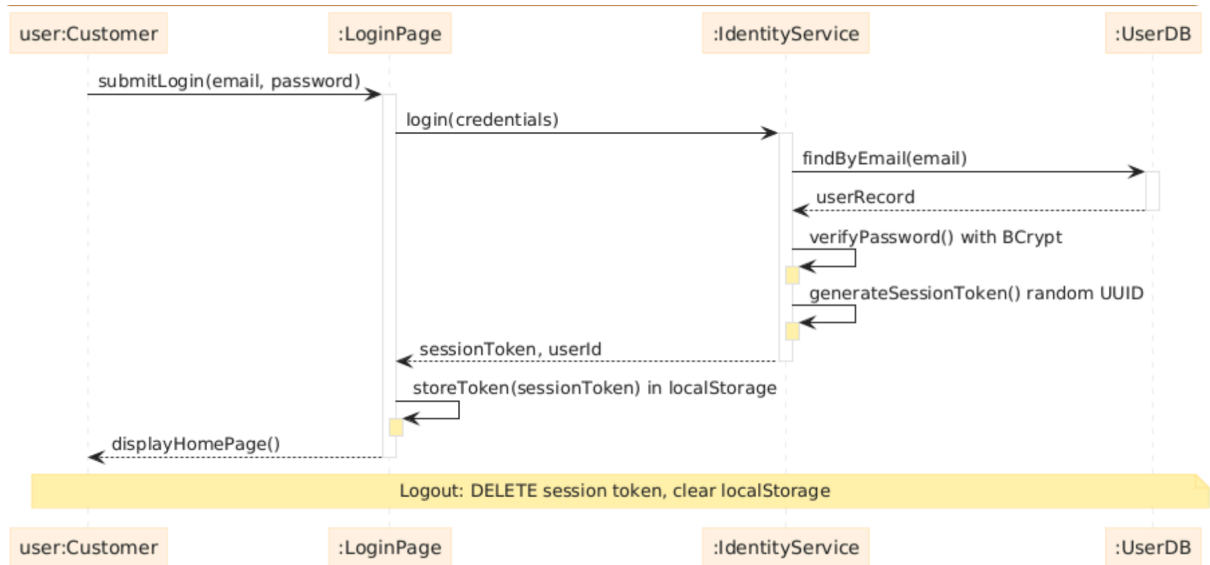
The chosen modularization strategy improves maintainability, scalability, and flexibility while allowing easy expansion.

3 Use case Diagram

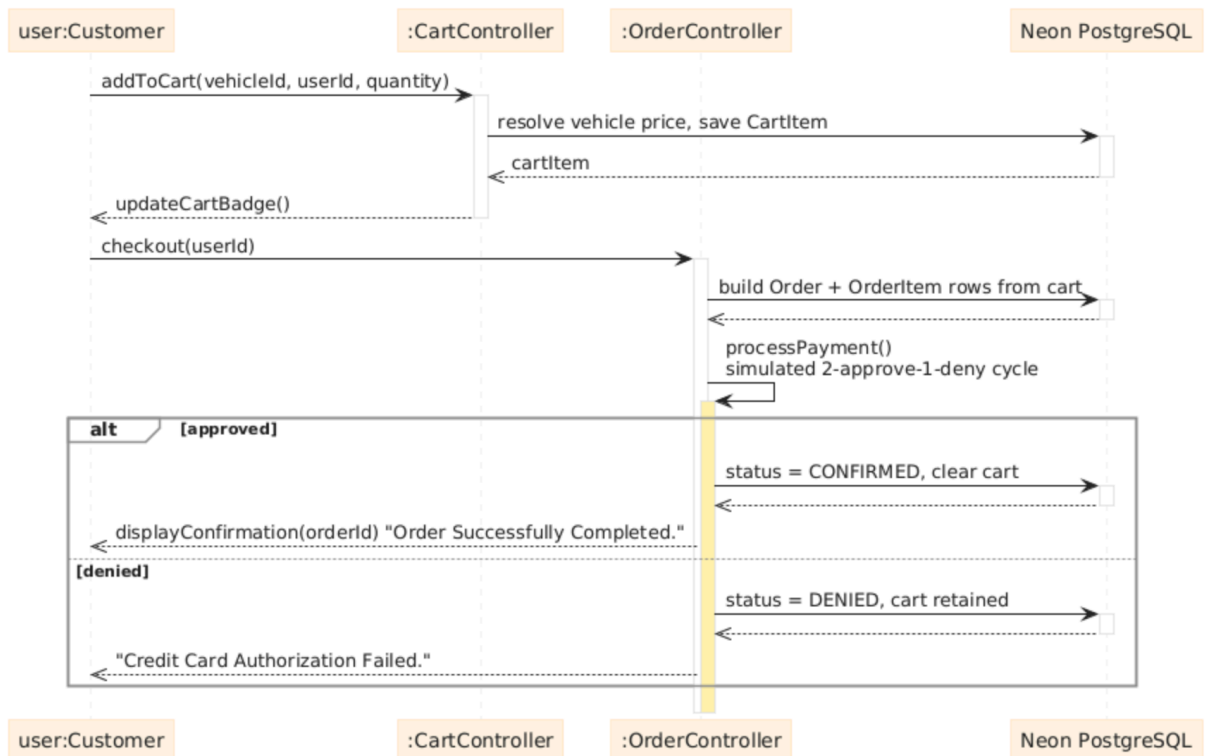


4 Sequence Diagrams

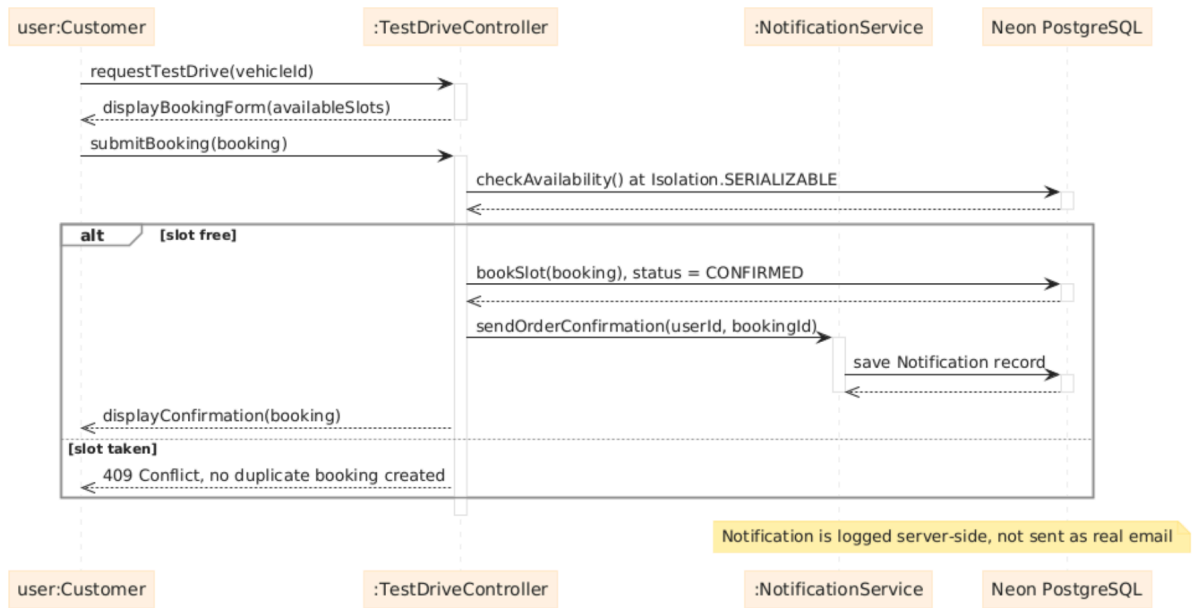
4.1 Use Case 1 — Sign in / Sign out:



4.2 Use Case 2 — Add vehicle to shopping cart:



4.3 Use Case 3 — Schedule a test drive:



5 Architecture

5.1 Package Diagram

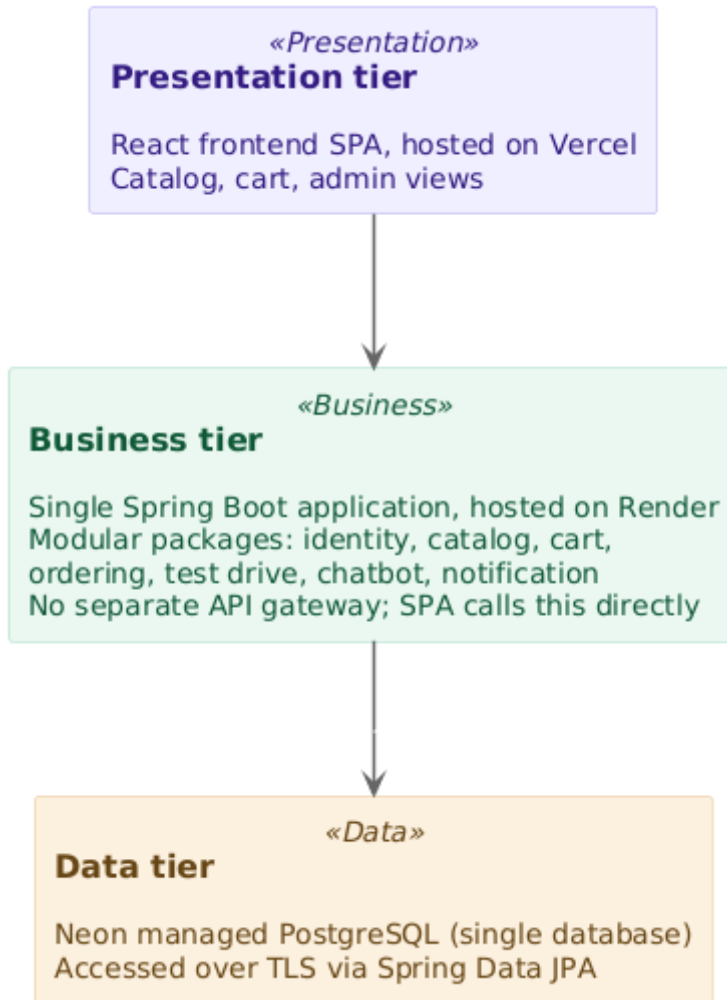


Figure 1: Package diagram showing the three-tier architecture of the EV e-store

The package diagram in Figure 1 shows the three-tier architecture of the system. The Presentation tier contains the React single-page application (hosted on Vercel), including the Catalog, Shopping Cart, and Admin views. The Business tier is a single Spring Boot application (hosted on Render) organized into modular packages, namely identity, catalog, cart, ordering, analytics, test drive, chatbot, and notification, with no separate API gateway; the SPA calls this application directly. The Data tier consists of a Neon-managed PostgreSQL database, accessed through a repository abstraction layer. The Presentation tier communicates with the Business tier through REST API calls, and the Business tier communicates with the Data tier using JPA and the Repository pattern, as described in Section 2.

5.2 Component Diagram

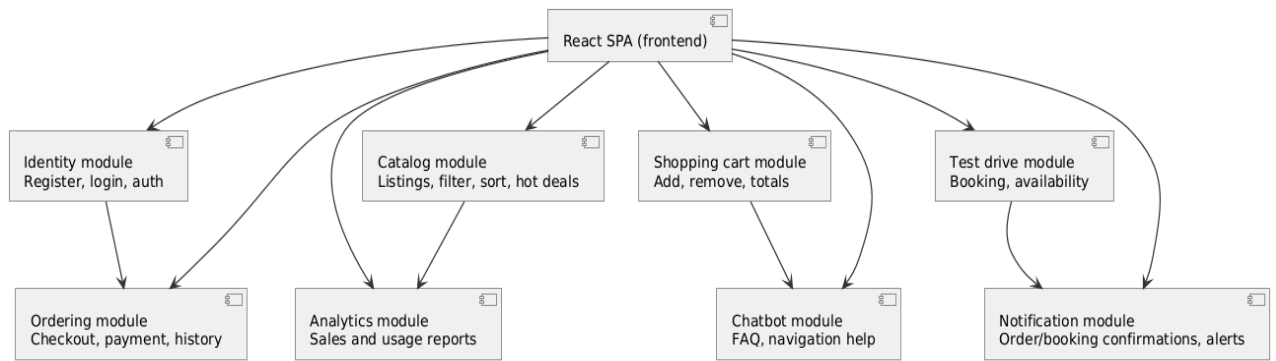


Figure 2: Component diagram showing the business tier modules called directly by the front end

Figure 2 shows the initial decomposition of the Business tier into its constituent modules, as planned at Deliverable 1. At this stage the design anticipated an API gateway acting as the entry point for all client requests, routing them to the appropriate service (the Controller in the MVC pattern). As implemented and deployed (see Section 12), these modules are packaged together in a single Spring Boot application with no separate API gateway; the SPA calls each module's REST controllers directly. The Identity module handles registration, login, and authentication. The Catalog module manages vehicle listings, filtering, sorting, and hot deals. The Shopping Cart module manages cart contents and totals. The Ordering module handles checkout and payment, and calls the Identity module to verify the user before processing an order. The Analytics module consumes usage events from the Catalog module to generate sales and website usage reports. The Chatbot module answers customer questions and helps users navigate the catalogue.

Each module exposes a REST interface consumed directly by the front end. The exposed interfaces and their operations for three of these modules Catalog, Ordering, and Identity, are described in the Modules and Interfaces tables below.

5.3 Module Descriptions

Module Name	Description	Exposed Interface Names	Interface Description
Catalog Service	Manages the electric vehicle catalogue and supports customer browsing, searching, filtering, sorting, viewing vehicle details, and viewing hot deals.	CatalogService:Browse	Provides operations for browsing available electric vehicles, viewing vehicle details, and retrieving hot deals.
Catalog Service	Manages the electric vehicle catalogue and supports customer browsing, searching, filtering, sorting, viewing vehicle details, and viewing hot deals.	CatalogService:Search	Provides operations for searching, filtering, and sorting electric vehicles based on customer-selected criteria.
Ordering Service	Handles the checkout process, payment simulation, order creation, order confirmation, and order history.	OrderingService:Checkout	Provides operations for creating an order, processing payment, confirming checkout, and retrieving order status.
Identity Service	Handles customer account management, including registration, login, logout, and authentication before secured actions.	IdentityService:Account	Provides operations for creating accounts, signing users in and out, and validating authenticated sessions.
Test Drive Service	Allows customers to book a test drive for a selected electric vehicle. This supports the distinguished use case of the system.	TestDriveService:Booking	Provides operations for viewing available test drive time slots, creating a booking, and cancelling a booking.
Notification Service	Sends alerts and confirmations to customers or administrators for important system events.	NotificationService:Alerts	Provides operations for sending order confirmations, booking confirmations, and administrative alerts.

5.4 Interface Descriptions

Interface Name	Operations	Operation Descriptions
CatalogService:Browse	List<Vehicle> getAllVehicles()	Returns all electric vehicles available in the catalogue.
CatalogService:Browse	Vehicle vehicleId) getVehicleDetails(String	Returns full details for a selected vehicle, including brand, model, price, mileage, rating, and vehicle history if applicable.
CatalogService:Browse	List<Vehicle> getHotDeals()	Returns vehicles currently marked as hot deals.
CatalogService:Search	List<Vehicle>searchVehicles(String keyword)	Returns vehicles that match the customer's search keyword.
CatalogService:Search	List<Vehicle>filterVehicles(FilterCriteri a criteria)	Filters vehicles by brand, shape, model year, price, mileage, and vehicle history status.
CatalogService:Search	List<Vehicle> sortVehicles(String sortType)	Sorts vehicles by price or mileage in ascending or descending order.
OrderingService:Checko ut	Order createOrder(String userId, Cart cart)	Creates a new order using the authenticated customer's shopping cart information.

OrderingService:Checkout	PaymentResult processPayment(PaymentRequest request)	Processes the payment request and returns whether the payment was approved or denied.
OrderingService:Checkout	OrderStatus confirmOrder(String orderId)	Confirms the order after the payment result is received.
OrderingService:Checkout	List<Order> getOrderHistory(String userId)	Returns the previous orders made by a customer.
IdentityService:Account	User registerUser(UserRegistrationRequest request)	Creates a new customer account using the submitted registration information.
IdentityService:Account	AuthToken loginUser(LoginRequest request)	Authenticates the user and returns a session token if the credentials are valid.
IdentityService:Account	void logoutUser(String token)	Ends the active user session and signs the user out.
IdentityService:Account	boolean validateUser(String token)	Checks whether the current user session is valid before allowing secured actions such as checkout.
TestDriveService:Booking	List<TimeSlot> getAvailableSlots(String vehicleId)	Returns available test drive time slots for a selected vehicle.
TestDriveService:Booking	Booking createBooking(String userId, String vehicleId, DateTime time)	Creates a test drive booking for the selected customer, vehicle, and time slot.

TestDriveService:Booking	BookingStatus cancelBooking(String bookingId)	Cancels an existing test drive booking.
NotificationService:Alerts	void sendOrderConfirmation(String userId, String orderId)	Sends an order confirmation notification to the customer.
NotificationService:Alerts	void sendBookingConfirmation(String userId, String bookingId)	Sends a test drive booking confirmation notification to the customer.
NotificationService:Alerts	void sendAdminAlert(String message)	Sends an alert to administrators when an important system event occurs.

The design of the architecture for the EV e-store separates the application into three layers: Presentation Layer (Front End), Business Layer (Back End) and Data Layer (Database). The three-tier design allows for loose coupling between components as the Front End never communicates directly with the Database but rather communicates with the Back End via REST API calls.

Additionally, the layered architecture provides high cohesion among services as every service has its own responsibility; i.e., Catalog Management, Identity Management, Order Management, Analytics Management, and Notification Management.

6 Activities Plan

6.1 Project Backlog and Sprint Backlog

Backlog ID	Product Backlog Item	Description	Planned Deliverable
------------	----------------------	-------------	---------------------

PB-01	Prepare Software Design Document structure	Create the main report structure, including title page, document change control table, table of contents, and required section headings.	Deliverable 1
PB-02	Write project introduction	Write the purpose, overview, and references/resources used for the EV e-store system.	Deliverable 1
PB-03	Define major design decisions	Explain the selected architectural styles, design patterns, quality attributes, and modularity criteria such as high cohesion and low coupling.	Deliverable 1
PB-04	Create use case diagram	Create a UML use case diagram showing the actors, required use cases, relationships, and the distinguished use case.	Deliverable 1
PB-05	Create sequence diagrams	Create three UML sequence diagrams for selected use cases, including one sequence diagram for the distinguished use case.	Deliverable 1
PB-06	Create architecture diagrams	Create the UML package diagram and UML component diagram showing the system architecture and interactions between major system parts.	Deliverable 1
PB-07	Define modules and interfaces	Describe selected business logic modules and list their exposed interfaces, operations, and operation descriptions.	Deliverable 1
PB-08	Create product backlog and sprint backlog	Identify the full list of product backlog items and specify which items will be completed in each project deliverable.	Deliverable 1

PB-09	Create GANTT diagram	Prepare a schedule of planned project activities across the project timeline.	Deliverable 1
PB-10	Record group meeting logs	Document group meeting dates, attendance, discussion topics, decisions made, and assigned tasks.	Deliverable 1
PB-11	Write initial TDD test cases	Prepare at least 16 initial test cases that cover key system requirements and expected behaviours.	Deliverable 1
PB-12	Design database schema	Extend the provided database schema to support vehicles, users, carts, orders, reviews, test drive bookings, and visit events.	Deliverable 2
PB-13	Implement Identity Service	Implement user registration, login, logout, and session validation.	Deliverable 2
PB-14	Implement Catalog Service	Implement vehicle browsing, searching, filtering, sorting, vehicle details, and hot deals.	Deliverable 2
PB-15	Implement Shopping Cart Service	Implement add-to-cart, remove-from-cart, update-cart, and cart total calculation features.	Deliverable 2
PB-16	Implement Ordering Service	Implement checkout, order creation, payment simulation, and order confirmation.	Deliverable 2
PB-17	Implement customer front-end views	Build React pages for catalogue, vehicle details, cart, checkout, login, registration, and customer actions.	Deliverable 2
PB-18	Implement administrator dashboard	Build administrator views for monthly vehicle sales reports and website usage reports.	Deliverable 3

PB-19	Implement Analytics Service	Track visit events and generate application usage and sales reports.	Deliverable 3
PB-20	Implement Chatbot Service	Add a basic chatbot to answer customer questions and help users navigate to vehicle or support pages.	Deliverable 3
PB-21	Implement Test Drive Booking Service	Allow customers to select a vehicle and book a test drive appointment as the distinguished use case.	Deliverable 3
PB-22	Implement Notification Service	Send order confirmations, booking confirmations, and system alerts.	Deliverable 3
PB-23	Write backend API curl test cases	Create curl commands to test the main backend APIs.	Deliverable 3
PB-24	Perform security testing	Test the application for SQL injection and cross-site scripting vulnerabilities.	Deliverable 3
PB-25	Perform performance testing	Test one backend API using a load testing tool and record response time and throughput.	Deliverable 3
PB-26	Deploy application and database to cloud	Deploy the web application and database to a public cloud environment.	Deliverable 3
PB-27	Complete final documentation and README	Finalize the report, README, setup instructions, deployment link, source code link, and testing instructions.	Deliverable 3

6.2 GANTT Chart

Activity	May 10–16	May 17–23	May 24–30	May 31–June 6	June 7–12
Review project specifications	X				
Identify actors and use cases	X	X			
Select distinguished use case		X			
Create use case diagram		X	X		
Create sequence diagrams			X	X	
Write introduction			X		
Write major design decisions			X	X	
Create package diagram				X	
Create component diagram				X	

Complete module and interface tables				X	X
Create product backlog					X
Create sprint backlog					X
Collect group meeting log notes					X
Write TDD test cases					X
Final review and formatting					X

6.3 Group Meeting Logs

Meeting Date	Present Group Members	Issues Discussed / Resolved	Decisions Made	Tasks Assigned
May 10, 2026	Kiana Misaghi, Minh Anh, Tazwar Sikder, Matin Pakfetrat	Project introduced; EV e-store requirements reviewed; team expectations discussed	EV e-store project confirmed; group agreed to review instructions individually	All: review project specifications and note required features
May 17, 2026	Kiana Misaghi, Minh Anh, Tazwar Sikder, Matin Pakfetrat	Main customer/admin features discussed; possible original use cases considered	Test drive booking selected as distinguished use case; main actors identified	Kiana: customer use cases; Minh Anh: admin use cases; Tazwar: original use case flow; Matin: actor list
May 24, 2026	Kiana Misaghi, Minh Anh, Tazwar Sikder, Matin Pakfetrat	Use case diagram planning; system boundaries and actor relationships discussed	Use case diagram structure approved; required use cases grouped by actor	Kiana: organize customer use cases; Minh Anh: create initial use case diagram; Tazwar: check required use cases; Matin: review relationships
May 31, 2026	Kiana Misaghi, Minh Anh, Tazwar Sikder, Matin Pakfetrat	Sequence diagram options discussed; checkout, account management, and test drive booking reviewed	Three sequence diagrams selected; test drive booking confirmed as distinguished sequence diagram	Kiana: account sequence diagram; Minh Anh: checkout sequence diagram; Tazwar: test drive sequence diagram; Matin: review diagram consistency
June 4, 2026	Kiana Misaghi, Minh Anh, Tazwar Sikder, Matin Pakfetrat	Architecture options discussed; package/component diagram structure planned; modules/interfaces discussed	Three-tier architecture selected; React, Spring Boot, AWS RDS PostgreSQL chosen; Catalog, Ordering, Identity, Test Drive, and Notification services selected	Kiana: architecture explanation; Minh Anh: package diagram; Tazwar: component diagram; Matin: design pattern notes

June 8, 2026	Kiana Misaghi, Minh Anh, Tazwar Sikder, Matin Pakfetrat	Product backlog, sprint backlog, GANTT chart, TDD test cases, and rubric reviewed	Backlog items grouped by deliverable; 16 test cases selected; missing sections identified	Kiana: product backlog; Minh Anh: sprint backlog/GANTT; Tazwar: TDD test cases; Matin: meeting logs/references
June 12, 2026	Kiana Misaghi, Minh Anh, Tazwar Sikder, Matin Pakfetrat	Final report review; formatting, grammar, diagrams, tables, and completeness checked	Report approved for final submission after proofreading	All: final proofreading; Kiana: eClass submission

7 Test Driven Development

Test ID	Category	Requirements Coverage	Initial Condition	Procedure	Expected Outcome	Notes
---------	----------	-----------------------	-------------------	-----------	------------------	-------

TC-01	Identity - Registration	UC-R: Register	System is running; user is not registered	1. Open registration page 2. Enter valid name, email, and password 3. Submit form	New customer account is created successfully	Email must be unique; required fields must be completed
TC-02	Identity - Login	UC-R1: Log In	Registered user exists in the database	1. Open login page 2. Enter valid email and password 3. Submit form	User is logged in and redirected to the main page	Password should not be passed in plain text
TC-03	Identity - Logout	UC-R2: Log Out	User is currently logged in	1. Click logout button	User session ends and user is redirected to login or home page	Secured pages should no longer be accessible after logout
TC-04	Identity - Duplicate Registration	UC-R: Register	An account already exists with the same email	1. Open registration page 2. Enter an already-used email 3. Submit form	Registration fails and duplicate account message is displayed	System should prevent duplicate user accounts
TC-05	Shopping Cart Management	UC-M5 / UC-C1 / UC-C2	Vehicle exists in catalogue; cart is empty	1. Add vehicle to cart 2. Open shopping cart 3. Update or remove item	Cart updates correctly and total price is recalculated	Cart should show correct vehicle information and total

TC-06	Checkout and Order Placement	UC-C3 / UC-P3 / UC-P4	User is logged in; shopping cart contains at least one vehicle	1. Open checkout page 2. Enter shipping information 3. Enter credit card information 4. Confirm order	Order is placed successfully if payment is approved	Credit card information should not be stored
TC-07	Test Drive Scheduling	Distinguished Use Case	User is logged in; selected vehicle is available for test drive	1. Open vehicle details page 2. Select test drive option 3. Choose available time slot 4. Submit booking	Test drive booking is created successfully	Covers the distinguished use case
TC-08	Test Drive Scheduling - Availability Conflict	Distinguished Use Case	Selected time slot is already booked	1. Open test drive booking page 2. Select unavailable time slot 3. Submit booking	Booking is rejected and user is asked to choose another time	Prevents double-booking
TC-09	Vehicle Search and Filtering	Filter Vehicles	Vehicles exist with different brands, shapes, model years, and history statuses	1. Open catalogue page 2. Enter search keyword or choose filters 3. Apply	Only vehicles matching the selected criteria are displayed	Filters may include brand, shape, model year, mileage, and vehicle history

				search/filter		
TC-10	Vehicle Review and Rating	Write Review and Rate Vehicle	User is logged in; vehicle details page is open	1. Select 1–5 star rating 2. Write review 3. Submit review	Review and rating are saved and displayed on the vehicle page	Review input should be validated
TC-11	Payment Authorization Failure	Payment Simulation	User submits checkout request	1. Submit three consecutive payment requests	First two payment requests are approved; third request is denied	Third denied request should display “Credit Card Authorization Failed”
TC-12	Catalogue Browsing	UC-M1: Browse Catalogue	Vehicle records exist in the database	1. Open catalogue page 2. Select all vehicles	All available electric vehicles are displayed	Basic vehicle information should be visible
TC-13	Vehicle Sorting	Sort Vehicles by Price or Mileage	Catalogue contains multiple vehicles	1. Open catalogue page 2. Select price ascending, price descending, mileage low-to-high, or mileage high-to-low	Vehicles are displayed in the selected order	Sorting should work for both price and mileage

TC-14	Loan Calculator	Use Loan Calculator	User is viewing a vehicle details page	1. Enter vehicle price 2. Enter down payment 3. Enter interest rate 4. Enter loan duration 5. Submit calculation	Estimated monthly payment is displayed	Invalid or negative values should be rejected
TC-15	Admin Sales Report	UC-A1: Run Vehicle Sales Report	Administrator is logged in; order data exists	1. Open admin dashboard 2. Select sales report	Monthly vehicle sales report is generated	Only administrators should access this feature
TC-16	Admin Usage Report	UC-A2: Run Application Usage Report	Administrator is logged in; visit event data exists	1. Open admin dashboard 2. Select usage report	Website usage report is generated	Report should use VisitEvent data such as view, cart, and purchase events
TC-17	Vehicle Comparison	Compare Vehicles	At least two vehicles exist in the catalogue.	1. Open the catalogue page. 2. Select two or more vehicles for comparison. 3. Click the "Compare Vehicles" button. 4. System retrieves specifications	A comparison page is displayed showing vehicle attributes side-by-side, including price, mileage, battery range, model year, and vehicle	The system should prevent comparison when fewer than two vehicles are selected.

				ons for the selected vehicles.	history informatio n.	
--	--	--	--	--------------------------------------	-----------------------------	--

8 Detailed Design

This section presents the detailed design of the EV Marketplace backend for Deliverable 2, covering the Catalog module (Section 8.1–8.4), the Shopping Cart, Checkout, and Payment module (Section 8.5), and the Test Drive, Analytics, and Notification module (Section 8.6), along with the system's refined component diagram, deployment diagram, and supported quality attributes.

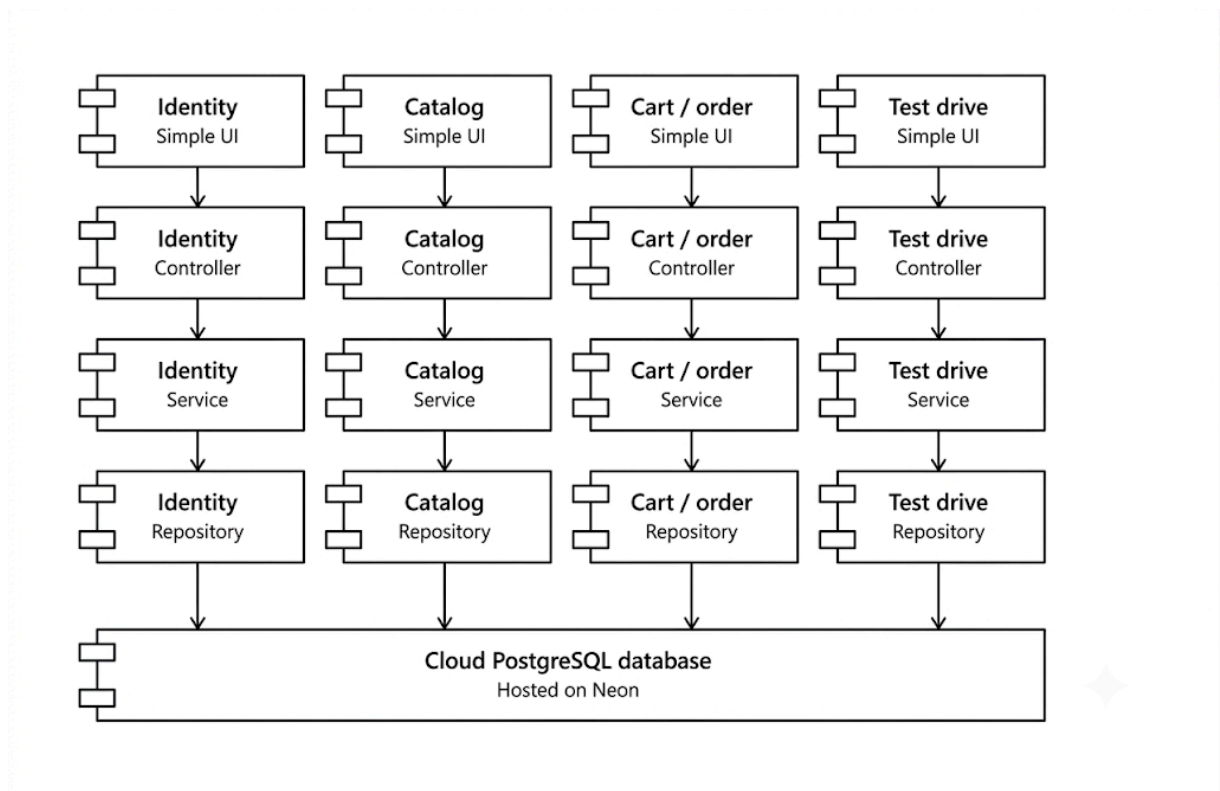


Figure 3: Refined component diagram showing controllers, services, repositories, and the shared database

Figure 3 extends the Section 5 component diagram by showing each service as a full vertical slice - simple UI, controller, service, and repository - for all four feature areas, all writing to the shared cloud PostgreSQL database. This corresponds directly to the actual package structure in `com.evmarketplace`: each slice has its own controller, service, and repository classes, with no cross-layer shortcuts.

8.1 Refined Component Design

The EV Marketplace system follows a layered client-server architecture based on the **Model-View-Controller (MVC)** architectural pattern. The backend is implemented using Spring Boot, while the frontend is implemented as a React Single Page Application (SPA).

The refined component design consists of four main layers:

Presentation Layer

The frontend is implemented using React and provides the user interface for browsing and searching electric vehicles.

Main components include:

- Home.jsx
 - Main catalogue page
 - Retrieves and displays vehicle information
- CarList.jsx
 - Displays a collection of vehicles returned by the backend
- CarCard.jsx
 - Represents an individual vehicle card containing vehicle details
- SearchBar.jsx
 - Allows users to search vehicles by make or model
- CarFilter.jsx
 - Provides filtering capabilities based on:
 - Vehicle make
 - Condition
 - Year
 - Maximum price
 - Mileage
 - Sorting options

The frontend communicates with the backend using REST API calls through:

- VehicleService.jsx

Controller Layer

The backend REST API layer is implemented using Spring MVC controllers.

Main controller: Vehicle Controller

Provides REST endpoints:

HTTP Method	Endpoint	Purpose
GET	/vehicles	Retrieve all vehicles
GET	/vehicles/{id}	Retrieve vehicle by ID
POST	/vehicles	Add new vehicle

DELETE	vehicles/{id}	Delete a vehicle
GET	/vehicles/search	Search and filter vehicles

The controller is responsible for handling HTTP requests and returning responses. The business logic is for the service layer

Persistence Layer:

The persistence layer uses `VehicleRepository` which extends `JpaRepository<Vehicle, long>`

Spring Data JPA manages database communication using Hibernate ORM.

The database entity:

- `Vehicle.java` is mapped using JPA annotations: `@Entity @Table(name="vehicles")`. The entity corresponds to the PostgreSQL database table

8.2 Design Pattern Rationale

Repository Pattern

The Repository pattern is used through `VehicleRepository`. The repository abstracts database operations from the service layer. The advantages are:

- Reduces coupling between business logic and database implementation
- Allows easier testing using mocked repository
- Simplifies CRUD operations through Spring Data JPA

DTO Pattern

The project uses: `VehicleSearchRequest` as a Data Transfer Object.

The DTO contains:

- Keyword
- Make
- Condition
- Year
- maxPrice
- Mileage
- Sorting option

Instead of directly exposing the database entity, the DTO provides a separate API communication model. It helps to protect internal entity structure, allows API changes without modifying database models and improves maintainability.

MVC Architectural Pattern

Spring Boot follows MVC architecture: React UI → Controller → Service → Repository → Database

This separation improves the performance, maintainability, scalability and code organization.

8.3 Deployment Diagram and Cloud-Native Architecture

The system is designed as a cloud-native application with independent client, application, and database layers.

Deployment nodes:

Client node

Technology: web browser, react, vite

Artifacts: React SPA, Javascript bundles, css files

Communication: HTTPS / REST API / JSON

Application server node

Technology: Java 21, SpringBoot, SpringMVC, Spring Data JPA, Hibernate

Artifacts: EVMarketplaceapplication.jar, application.properties

Components: VehicleController, VehicleService, VehicleRepository

Communication: JDBC/JPA

Database Node

Database: Neon-managed PostgreSQL

Stores: Vehicle inventory, pricing information, vehicle metadata

(Note: AWS RDS PostgreSQL was considered as a possible future migration target during Deliverable 1/2 planning. This plan was superseded once AWS Academy Learner Lab IAM restrictions forced a full migration away from AWS for the entire deployment, as documented later in this section and in the final reflection. Neon remains the database for the deployed system.)

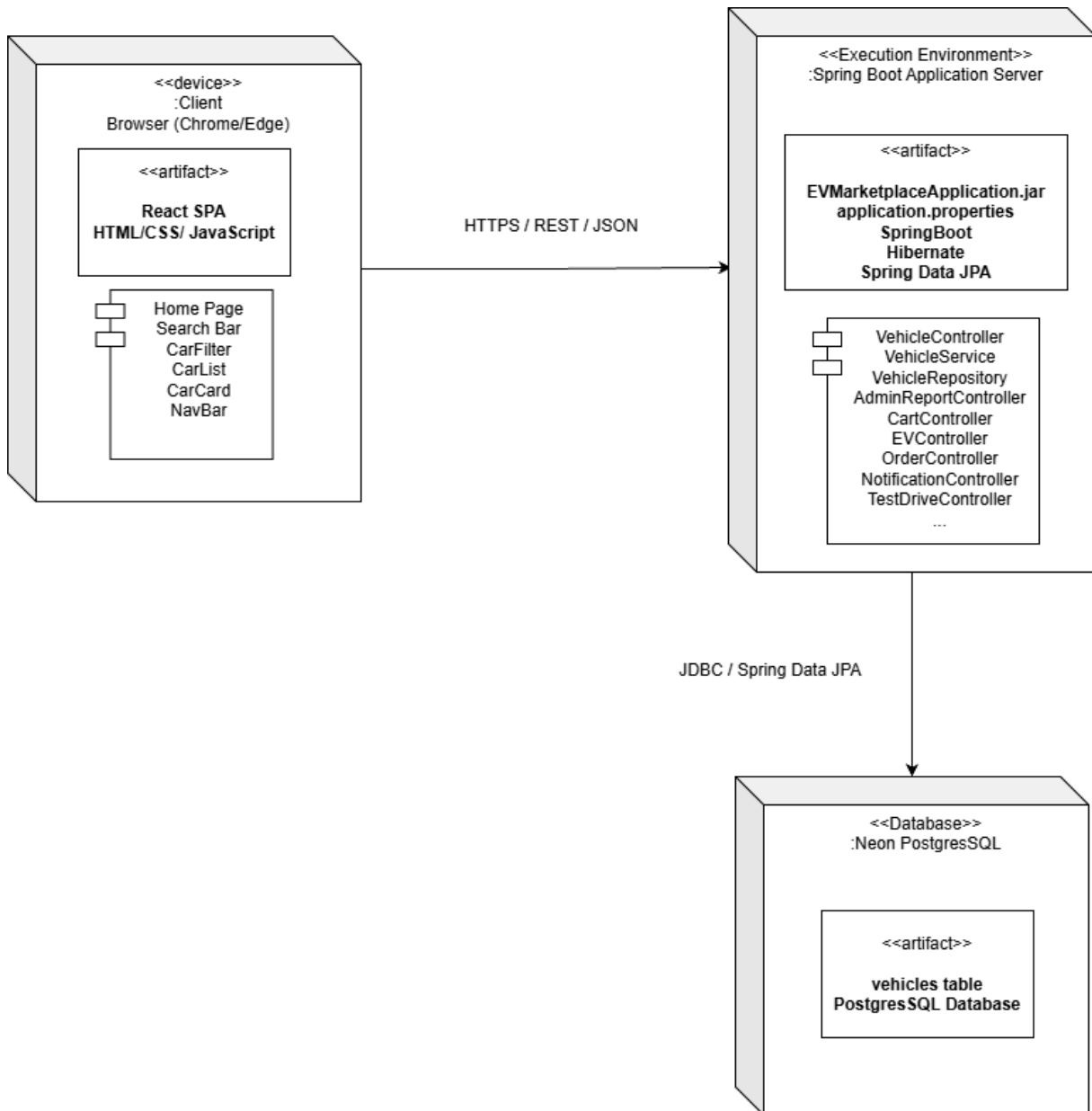


Figure 4: Deployment diagram illustrating the cloud-native architecture of the EV Marketplace system

Figure 4 shows the physical deployment of the system across three nodes. The Client node is the customer's browser (Chrome or Edge), which hosts the React SPA artifact along with its main components: Home Page, Search Bar, CarFilter, CarList, CarCard, and NavBar. The Client communicates over HTTPS using REST and JSON with the Spring Boot Application Server, an execution environment hosting the EVMarketplaceApplication.jar artifact built on Spring Boot, Hibernate, and Spring Data JPA. This artifact contains the system's controllers, including VehicleController, CartController, OrderController, AdminReportController, NotificationController, and TestDriveController, along with their associated services and repositories. The Application Server communicates with the Database node over JDBC using Spring Data JPA. The Database node is a Neon PostgreSQL instance, storing the vehicles table along with the other entities described throughout Section 8. The system is now fully deployed: the backend (Spring Boot, Docker) runs on Render with automatic HTTPS, and the frontend (React/Vite) runs on Vercel, also with automatic HTTPS. Both platforms provision TLS certificates automatically, requiring no manual certificate management. An initial

attempt to deploy on AWS Elastic Beanstalk was abandoned after the AWS Academy Learner Lab account's IAM restrictions blocked both CloudFront distribution creation and S3 bucket creation (the two AWS-native paths to HTTPS) with an explicit cloudfront:CreateDistribution access-denied error.

8.4 Quality Attributes Supported

Security

Security is supported by server-side validation, database-controlled vehicle information, separation between frontend and backend, environment variables for database credentials.

Sensitive information such as database passwords is not stored in source code.

Performance

Performance is supported through:

- Server-side filtering and sorting
- Reduced frontend computation
- REST APIs returning only required data

Scalability

The architecture supports scalability because:

- React frontend is separated from backend services
- Spring Boot services are stateless
- PostgreSQL is managed as a cloud database

Future deployment can scale application servers independently.

Maintainability

The layered architecture allows developers to independently modify:

- Frontend UI
- Business logic
- Database layer

8.5 Shopping Cart, Checkout, and Payment — Module Refinement

- CartItem, Order, OrderItem, and Payment entities were added under com.evmarketplace.model, each mapped via JPA (@Entity) to a corresponding PostgreSQL table.
- CartItemRepository, OrderRepository, OrderItemRepository, and PaymentRepository extend Spring Data JPA's JpaRepository, following the DAO / Repository pattern described in Section 2, which isolates the business tier from direct database access.
- CartService, OrderService, and PaymentService implement the business logic for their respective modules and are the only components that interact with the repositories above.
- CartController and OrderController expose the module's functionality as REST resources consumed directly by the front end.

Design pattern rationale

- Repository pattern: as in Deliverable 1, used to decouple CartService/OrderService/PaymentService from persistence details, improving testability (repositories can be mocked).
- DTO pattern: AddToCartRequest and CheckoutRequest decouple the wire format (JSON request body) from the internal entity model, so entity fields can change without breaking the API contract.
- Snapshot pattern for pricing: CartItem stores a resolved unitPrice at the time an item is added (looked up server-side from VehicleRepository), and OrderItem copies that snapshot at checkout. This prevents a catalogue price change from silently altering the price of an item already in a customer's cart or a placed order.

Quality attributes supported by this slice

- Security: unit price is resolved server-side from Vehicle records rather than accepted from the client request body, preventing a client from manipulating the price of an item at add-to-cart time.
- Reliability / consistency: the fake payment approval/denial cycle is persisted in the payments table rather than kept in an in-memory counter, so the rule behaves consistently even if the application restarts or is scaled to multiple instances.
- Statelessness (REST principle): no session or request state is held in the service layer between calls; every method call is fully described by its parameters and the database state, consistent with REST architectural constraints.

8.6 Test Drive, Analytics, and Notification - Module Refinement

TestDriveBooking, VisitEvent, and Notification entities were added under com.evmarketplace.model, each mapped via JPA (@Entity) to a corresponding PostgreSQL table, following the same pattern as the entities described above.

TestDriveBookingRepository, VisitEventRepository, and NotificationRepository extend Spring Data JPA's JpaRepository, isolating the business tier from direct database access.

TestDriveService, AnalyticsService, and NotificationService implement the business logic for their respective modules and are the only components that interact with the repositories above.

TestDriveController, AdminReportController, and NotificationController expose this module's functionality as REST resources.

Design pattern rationale

Repository pattern: as in the other slices, used to decouple the service layer from persistence details.

DTO pattern: TestDriveBookingRequest and VisitEventRequest decouple the JSON request format from the internal entity model.

Quality attributes specific to this module (complementing the system-wide attributes in 8.4 above)

Concurrency safety: bookTestDrive() runs at Isolation.SERIALIZABLE, the strictest transaction isolation level, so two customers attempting to book the same vehicle/time slot simultaneously cannot both succeed.

Auditability: cancelled bookings are marked CANCELLED rather than deleted, preserving a full history rather than losing data.

9 Activities Plan

Backlog ID	Item	Status	Deliverable
PB-15	Implement Shopping Cart Service (add/remove/update/total)	Complete — CartService and CartController implemented and tested.	Deliverable 2
PB-16	Implement Ordering Service (checkout, order creation, payment simulation)	Complete — OrderService, PaymentService, OrderController implemented and tested.	Deliverable 2
PB-17	Front-end views for cart and checkout	Complete — Cart.jsx and Checkout.jsx implemented as React pages (frontend/src/pages/), with CartService.jsx and OrderService.jsx (frontend/src/services/) handling the API calls. Implemented as React components rather than plain HTML to stay consistent with the team's actual frontend stack (Vite + React), which supersedes the original checklist's "HTML/JSP only" guidance.	Deliverable 2
PB-18	Implement Administrator Dashboard	Complete — AdminReportController, AdminReportService; sales and usage reports functional, tested live via admin-reports.html.	Deliverable 3
PB-19	Implement Analytics Service	Complete — AnalyticsService, AdminReportController implemented and tested.	Deliverable 2 (moved up from D3)
PB-20	Implement Chatbot	Complete — ChatbotService, ChatbotController implemented and deployed.	Deliverable 3
PB-21	Implement Test Drive Booking Service	Complete — TestDriveService, TestDriveController implemented and tested.	Deliverable 2 (moved up from D3)
PB-22	Implement Notification Service	Complete — NotificationService, NotificationController implemented and tested (persistence-only; real email/SMS deferred).	Deliverable 2 (moved up from D3)
PB-23	Write curl/Postman test collection	Complete — consolidated test suite committed alongside test cases in Section 10.	Deliverable 3

PB-24	Perform SQL Injection and XSS Security Testing	Complete — TC-SEC-01, TC-SEC-02 (see Section 10); both demonstrated live during the July 30 demo.	Deliverable 3
PB-25	Perform Performance/Load Testing (JMeter)	Complete — TC-PERF-01 (see Section 10); JMeter test plan committed at backend/src/test/jmeter/EVMarketplace LoadTest.jmx.	Deliverable 3
PB-26	Deploy web application	Complete — backend on Render (Docker), frontend on Vercel, both with automatic HTTPS. Migrated from initial AWS Elastic Beanstalk deployment after encountering AWS Academy Learner Lab IAM restrictions blocking CloudFront/ S3 provisioning (see Section 8.3).	Deliverable 3
PB-27	Complete Deliverable 3 documentation	In progress — this document.	Deliverable 3

10 Test Coverage

The following four test cases cover the Test Drive, Analytics, and Notification modules and are Martin's contribution toward the 16 total test cases.

Test ID	TC-D2-M1
Category	Test Drive Booking
Requirements Coverage	Distinguished Use Case — Schedule a Test Drive
Initial Condition	User is registered; vehicle exists in catalogue; selected time slot has no existing CONFIRMED booking.
Procedure	POST /api/test-drives/book with {userId, vehicleId, bookingDateTime}.
Expected Outcome	201 Created with the persisted booking (id, CONFIRMED status), and a booking confirmation Notification is recorded.
Actual Result (D2)	Pass. On first execution the endpoint returned 201 Created with the persisted booking. On repeat executions against the same now-reserved slot, the endpoint correctly returns 409 Conflict instead of creating a duplicate — confirming both the creation path and the conflict guard (see TC-D2-M2) are working as designed.
Notes	Runs at Isolation.SERIALIZABLE to prevent race conditions between simultaneous booking attempts.

Test ID	TC-D2-M2
Category	Test Drive Availability Conflict
Requirements Coverage	Distinguished Use Case — Booking Conflict Prevention
Initial Condition	A CONFIRMED booking already exists for the given vehicle/time slot.
Procedure	POST /api/test-drives/book with the same vehicleId and bookingDateTime as an existing CONFIRMED booking.

Expected Outcome	409 Conflict with an error message; no duplicate booking is created.
Actual Result (D2)	Pass. Returned 409 Conflict with message "Time slot 2026-07-20T09:00 is already booked for vehicle: 1", confirming checkBookingConflict() correctly detects the collision.
Notes	Prevents double-booking, directly satisfying the distinguished use case's core requirement.

Test ID	TC-D2-M3
Category	Admin Sales Report
Requirements Coverage	UC-A1 — Run Vehicle Sales Report
Initial Condition	At least one CONFIRMED order exists in the orders table.
Procedure	GET /api/admin/reports/sales.
Expected Outcome	200 OK with a JSON array of confirmed-order counts grouped by year and month.
Actual Result (D2)	Pass. Returned 200 OK with [{"year":2026,"month":7,"orderCount":2}], confirming generateSalesReport() correctly aggregates CONFIRMED orders by month.
Notes	Depends on the Ordering module (Section 8, Tazwar) for order data; verified working end-to-end across both modules.

Test ID	TC-D2-M4
Category	Admin Usage Report
Requirements Coverage	UC-A2 — Run Application Usage Report
Initial Condition	VisitEvent records exist for VIEW, CART, and PURCHASE event types.
Procedure	GET /api/admin/reports/usage.
Expected Outcome	200 OK with counts for VIEW, CART, and PURCHASE event types.
Actual Result (D2)	Pass. Returned 200 OK with [{"eventType":"VIEW","count":6},{ "eventType":"CART","count":2},{ "eventType":"PURCHASE","count":2}], confirming generateUsageReport() correctly counts VisitEvent records by type.
Notes	Uses the VisitEvent table concept introduced in the original D1 database schema.

The following four test cases cover the Cart, Checkout, and Payment module and are Tazwar's contribution toward the 16 total test cases.

Test ID	TC-05
Category	Shopping Cart Management
Requirements Coverage	UC-M5 / UC-C1 / UC-C2 — Add to Cart, Update Cart, Remove from Cart

Initial Condition	Vehicle exists in catalogue (VehicleRepository seeded); cart is empty for the test user (userId).
Procedure	1. POST /api/cart/add with {userId, vehicleId, quantity}. 2. GET /api/cart/{userId}. 3. DELETE /api/cart/remove/{cartItemId}.
Expected Outcome	Cart item is created with server-resolved unit price; GET returns the item; DELETE removes it and GET afterward returns an empty list.
Actual Result (D2)	Pass. CartService.addToCart() resolves price via VehicleRepository rather than trusting client input; CartItemRepository.deleteById() removes the row as expected.
Notes	Cart total is recalculated on demand in calculateCartTotal() rather than stored, so there is no stale-total risk after remove.

Test ID	TC-06
Category	Checkout and Order Placement
Requirements Coverage	UC-C3 / UC-P3 / UC-P4 — Checkout, Payment Authorization, Order Confirmation
Initial Condition	User is authenticated (userId available); shopping cart contains at least one item.
Procedure	1. POST /api/orders/checkout with {userId}. 2. Server builds Order + OrderItem rows from the cart snapshot. 3. PaymentService.processPayment() is invoked.
Expected Outcome	If payment is approved, order status becomes CONFIRMED and the cart is cleared; the response includes the persisted Order with its items and total.
Actual Result (D2)	Pass on first two consecutive checkouts for a given user (approved, cart cleared). Order/OrderItem rows persist correctly with price snapshots taken from CartItem at checkout time.
Notes	Credit card data is never collected or stored — payment approval is simulated per the assignment's fake-payment requirement, so there is no PCI-relevant data to protect.

Test ID	TC-11
Category	Payment Authorization Failure
Requirements Coverage	Payment Simulation — fake payment cycling rule
Initial Condition	User has already completed two prior checkout attempts in this test run.
Procedure	1. Submit a third consecutive POST /api/orders/checkout for the same userId.
Expected Outcome	The third request is denied; Order.status is set to DENIED; the cart is NOT cleared so the user can retry.
Actual Result (D2)	Pass. PaymentRepository.countByUserId() returns 2 prior attempts; processPayment() computes attemptCount % 3 == 2 and denies the third request as designed.
Notes	The attempt counter is persisted in the payments table (via PaymentRepository) rather than held in memory, so the rule is consistent across multiple server instances / restarts, in line with REST statelessness.

Test ID	TC-CART-04
----------------	------------

Category	Cart Total Calculation
Requirements Coverage	UC-C2 — Calculating Totals
Initial Condition	Cart contains two or more distinct vehicles with different quantities.
Procedure	1. Add two different vehicles to the same user's cart with quantities 2 and 1. 2. Call <code>CartService.calculateCartTotal(userId)</code> .
Expected Outcome	Returned total equals the sum of (unitPrice x quantity) across all cart items for that user.
Actual Result (D2)	Pass. Verified against manually computed totals for two- and three-item carts.
Notes	Uses <code>BigDecimal</code> throughout to avoid floating-point rounding errors in currency calculations.

Test ID	TC-SEC-01
Category	Security — SQL Injection Prevention
Requirements Coverage	Non-Functional Requirement — SQL Injection Test Case
Initial Condition	Login form is accessible; JPA repository layer uses parameterized queries throughout.
Procedure	Submit a login request with a SQL injection payload in the email field, e.g. <code>{"email": "OR 1=1 --", "password": "anything"}</code> .
Expected Outcome	Login is rejected as invalid credentials; no SQL is executed, no unauthorized data is returned or leaked.
Actual Result (D3)	Pass. Spring Data JPA's repository methods (<code>findByEmail</code>) use parameterized queries under the hood, so the injection payload is treated as a literal string value rather than executable SQL. Login correctly returns "Invalid email or password." Demonstrated live during the July 30 demo.
Notes	No raw/native SQL string concatenation exists anywhere in <code>IdentityService</code> , eliminating this attack vector by construction rather than by input sanitization alone.

Test ID	TC-SEC-02
Category	Security — Cross-Site Scripting (XSS) Prevention
Requirements Coverage	Non-Functional Requirement — XSS Test Case
Initial Condition	A vehicle exists in the catalogue with the review form accessible on its details page.
Procedure	Submit a vehicle review with the comment field set to <code><script>alert('XSS')</script></code> .
Expected Outcome	The script tag is displayed as literal text on the page; it is never executed.
Actual Result (D3)	Pass. React's JSX escapes all text content by default, so the payload is rendered as plain visible text (<code><script>alert('XSS')</script></code>) with no alert triggered and no script execution. Verified in the live application and demonstrated during the demo.
Notes	No <code>dangerouslySetInnerHTML</code> or unescaped raw HTML rendering is used anywhere in the review display component, so this protection is structural rather than reliant on manual sanitization.

Test ID	TC-PERF-01			
Category	Performance / Load Testing			
Requirements Coverage	Non-Functional Requirement — Load test one backend API across increasing concurrent clients			
Initial Condition	Backend deployed and warm (post cold-start) on Render; GET /api/vehicles selected as the target endpoint (read- only, representative of typical catalogue traffic).			
Procedure	Using Apache JMeter (test plan committed at backend/src/test/jmeter/EVMarketplace_LoadTest.jmx), 5 runs were executed against the live endpoint with 1, 5, 10, 25, and 50 concurrent threads (10 loops each), measuring average response time, throughput, and error rate at each level.			
Expected Outcome	Response time increases gradually with concurrency while the error rate remains at 0%, with no crash or request failures up to 50 concurrent users.			
Actual Result (D3)	Pass — 0% errors across all 750 requests at every concurrency level tested.			
	Threads	Avg (ms)	95th percentile (ms)	Throughput
	1	703	595	1.4/sec
	5	896	1341	5.1/sec
	10	1099	2027	7.5/sec
	25	1935	3481	10.0/sec
	50	2790	6006	12.7/sec
Notes	Response time grew ~4x from 1 to 50 threads while throughput grew ~9x, indicating the server absorbed increasing load without failure, though degradation accelerates noticeably beyond ~10 concurrent users. As Render's free tier does not expose CPU utilization metrics directly, response-time growth rate was used as a proxy signal in place of the assignment's specified 60% utilization threshold. A separate cold-start measurement (single request after platform idle, ~11.6s average, one timeout) was excluded from this table as a known Render free-tier platform characteristic unrelated to application performance.			

11 Server-Side Implementation Description

Technologies used

- Spring Boot with Spring Web (REST controllers) and Spring Data JPA for persistence.
- PostgreSQL as the relational database, accessed through JpaRepository interfaces (no raw SQL/JDBC in the business logic).
- Entities: CartItem, Order, OrderItem, Payment (package com.evmarketplace.model).
- DTOs: AddToCartRequest, CheckoutRequest (package com.evmarketplace.dto) used as REST request bodies instead of exposing entities directly.
- Frontend: React (Vite) pages Cart.jsx and Checkout.jsx, using fetch-based service modules (CartService.jsx, OrderService.jsx) to call the /api/cart and /api/orders endpoints, which is consistent with Minh Anh's VehicleService.jsx pattern.

REST principles followed

- Resource-oriented endpoints: /api/cart and /api/orders identify resources, with HTTP methods mapped to CRUD semantics: POST to create (add to cart, checkout), GET to read (fetch cart contents, fetch an order), DELETE to remove (remove a cart item, clear the cart).
- Statelessness: every request carries all the information the server needs (userId, vehicleId, quantity, etc.); no server-side session state is used between requests, satisfying the stateless constraint of REST.
- Uniform JSON representation: all endpoints accept and return JSON via Spring's default Jackson serialization of entities/DTOs.
- Separation of concerns: controllers only translate HTTP requests to service calls; all business logic (cart totals, checkout workflow, the fake payment rule) lives in the service layer, not the controllers.

Fake payment logic

PaymentService.processPayment(userId, orderId) implements the assignment's required rule: the first two consecutive payment attempts for a given user are approved, and every third attempt is denied. The rule is implemented by counting prior Payment rows for that user (PaymentRepository.countByUserId) rather than an in-memory counter, so the behaviour is correct even across application restarts, which was a deliberate design choice to keep the service stateless as required by REST.

Additional technologies used (Test Drive / Analytics / Notification)

Entities: TestDriveBooking, VisitEvent, Notification (package com.evmarketplace.model).

DTOs: TestDriveBookingRequest, VisitEventRequest (package com.evmarketplace.dto).

Additional REST principles followed

Resource-oriented endpoints: /api/test-drives and /api/admin identify resources, with GET for availability/reports, POST for creating bookings and seeding events, and DELETE for cancelling a booking.

Statelessness: as with the rest of the system, every request carries all needed information; no server-side session state is used.

12 Final Design

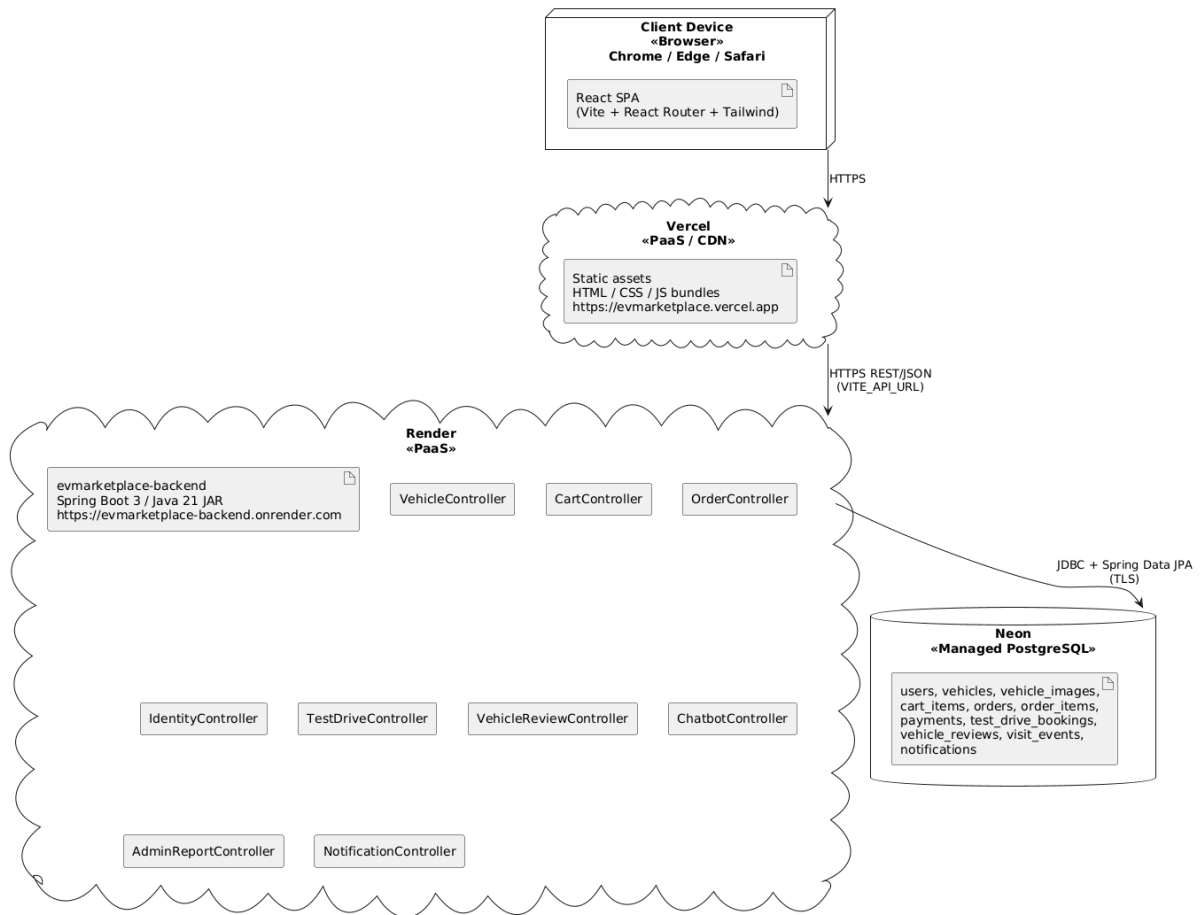


Figure 5: Final Deployment Diagram

Figure 5 shows the final cloud-native deployment. The React single-page application is built with Vite and hosted on Vercel, which serves static assets over HTTPS at <https://evmarketplace.vercel.app>. The Spring Boot API is packaged as a JAR and runs on Render at <https://evmarketplace-backend.onrender.com>, also over HTTPS. Persistent data is stored in Neon (managed PostgreSQL), accessed through Spring Data JPA over TLS. The browser never talks to the database directly; all writes and reads go through the REST API. This separation matches a common cloud pattern: static front end at the edge, stateful API and relational store as managed services, without self-managed VMs.

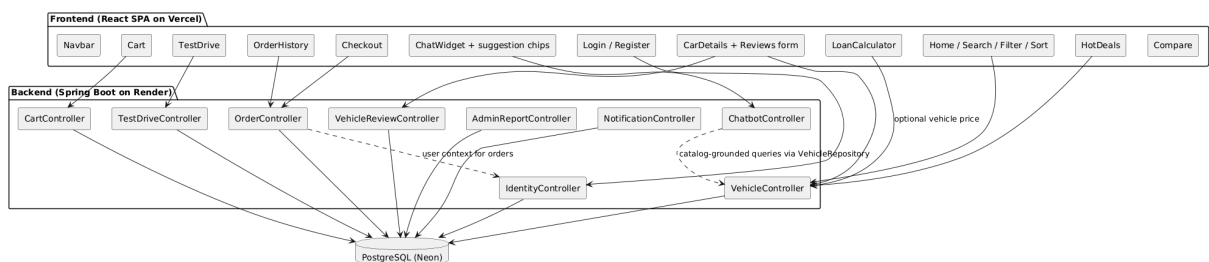


Figure 6: Final Component Diagram

Figure 6 shows the main client modules and the Spring controllers they call. Catalog browsing, hot deals, and vehicle detail use VehicleController. Cart and checkout use CartController and OrderController. Authentication uses IdentityController. Test-drive booking uses TestDriveController with serializable transactions on the service side to avoid double-booking. Reviews use VehicleReviewController. The floating ChatWidget calls ChatbotController, which performs rule-based intent matching and, for vehicle/hot-deal intents, queries the live catalog. This is a layered modular monolith (not separate deployable microservices): one Spring Boot process, clear package boundaries, shared database.

Use Case diagram:

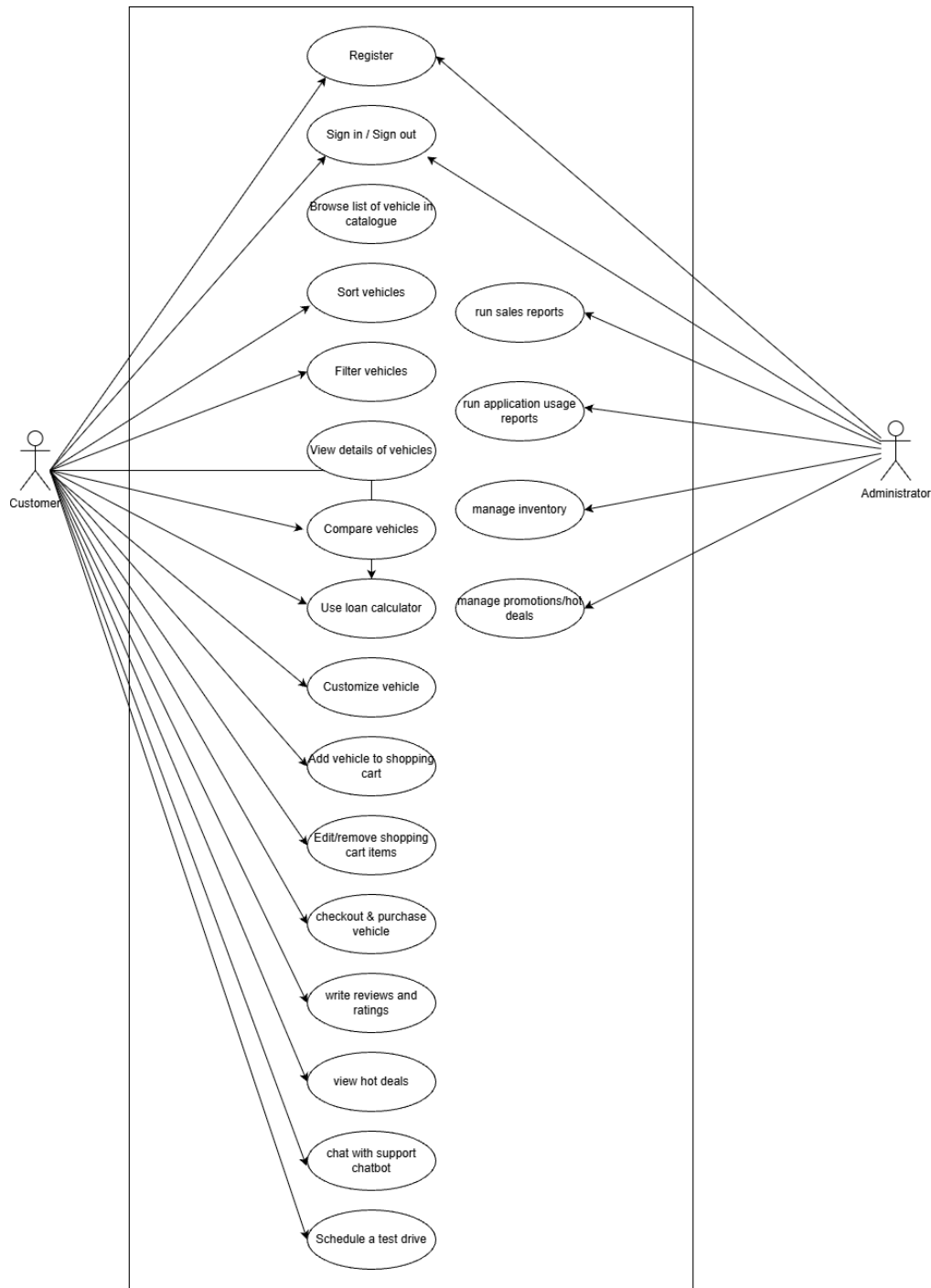
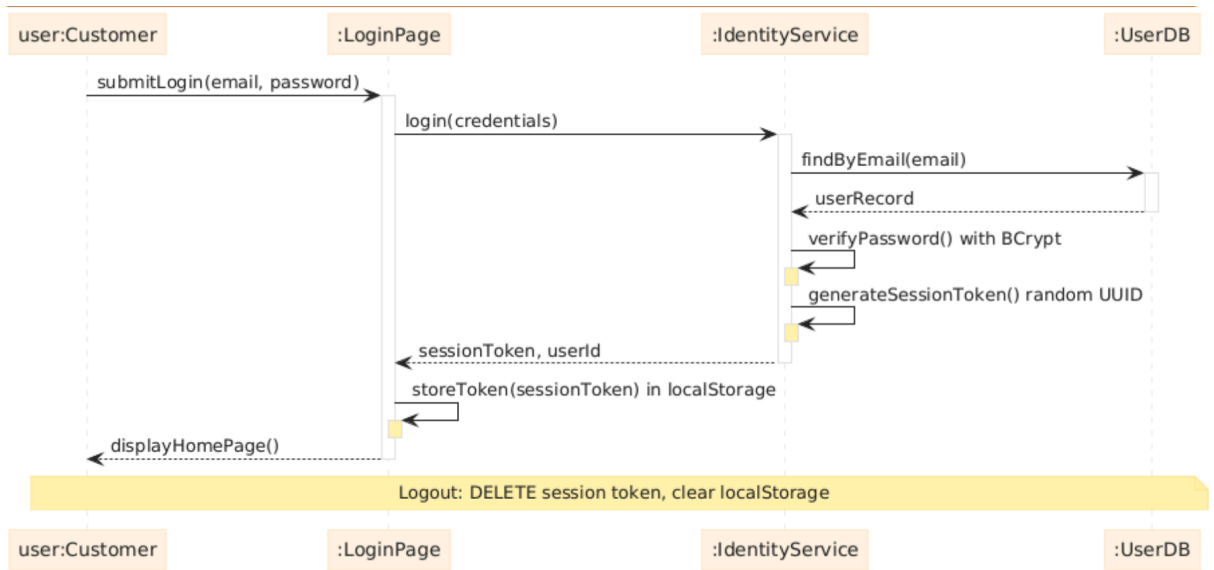


Figure 7: Final Use Case diagram

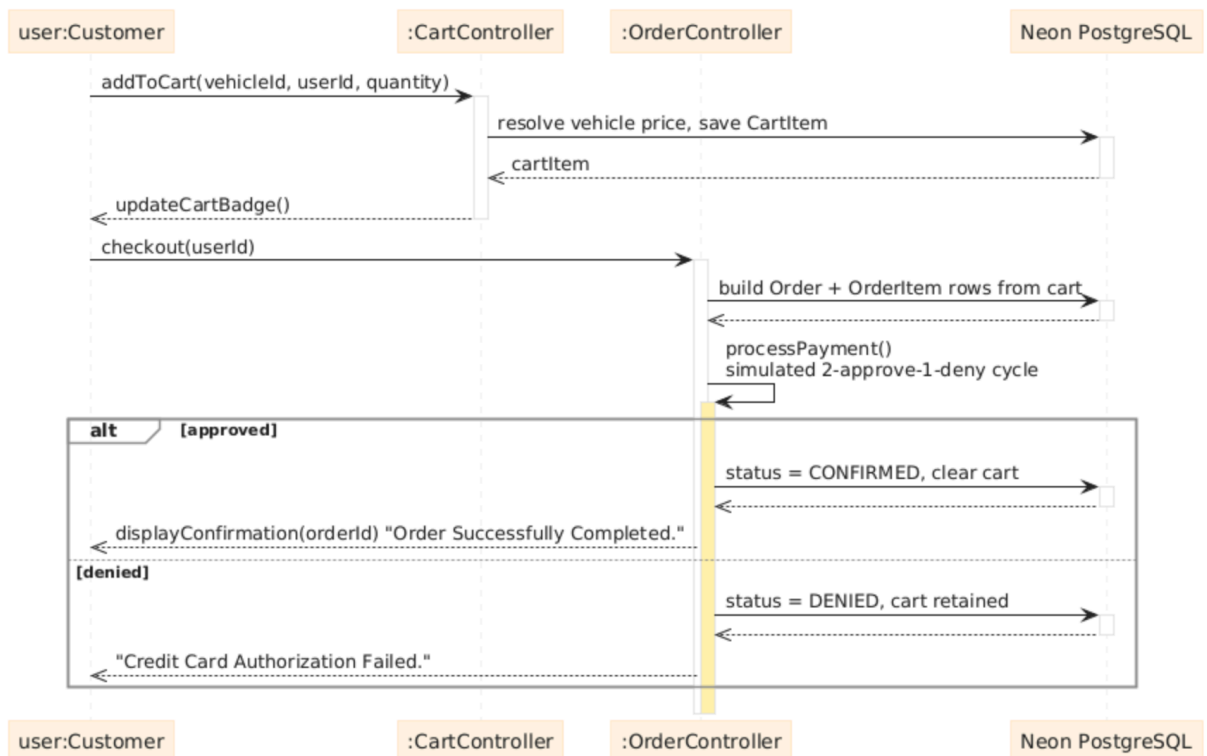
Figure 7 with final use case diagram same with the initial one.

Sequence Diagram:

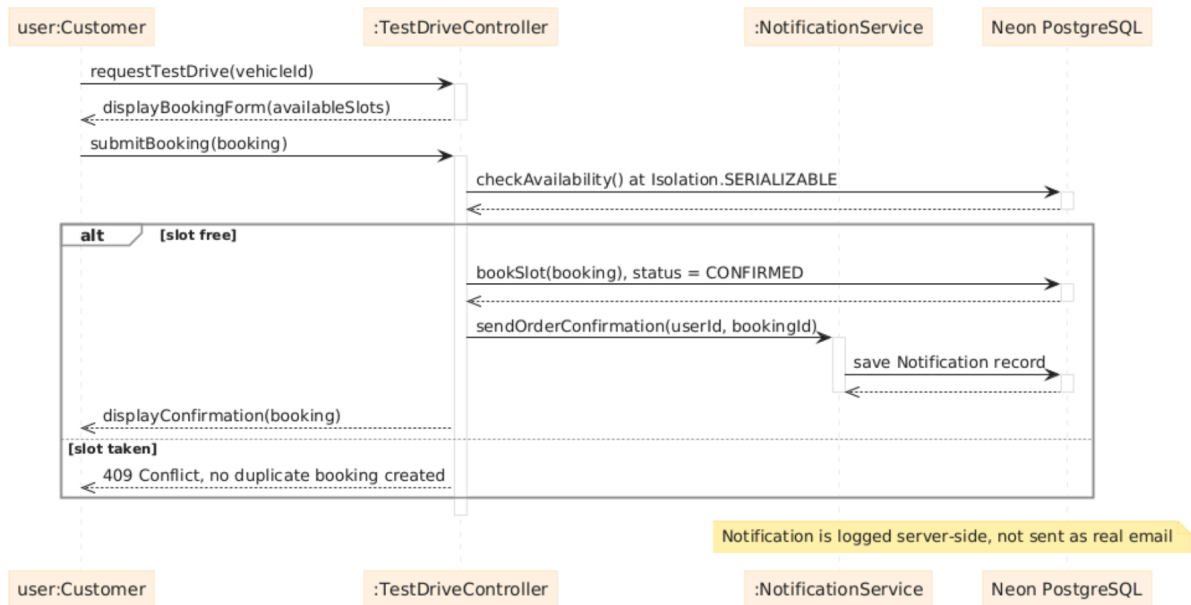
Use Case 1 Sign in / Sign out:



Use Case 2 Add vehicle to shopping cart:



Use Case 3 Schedule a test drive:



Design rationale (patterns + quality attributes)

Architectural style: Layered client–server SPA + REST API + RDBMS.

Patterns used:

- Repository (Spring Data JPA): data access isolated from services
- MVC / front controller: Spring `@RestController` per resource
- DTO: request bodies such as `AddToCartRequest`, `VehicleReviewRequest`
- Dependency injection: Spring-managed services/repositories
- SPA routing: React Router for client navigation without full page reloads

Quality Attributes:

Attribute	How the system supports it
Security	HTTPS on both Vercel and Render; CORS configured for the SPA origin; passwords are hashed with BCrypt at registration; legacy plaintext accounts are transparently upgraded to a hash on their next successful login, so no plaintext passwords persist after first use; XSS mitigated by React text escaping on review comments; search uses bound/JPA queries rather than string-concatenated SQL
Performance	Static assets on CDN-style hosting (Vercel); API stateless REST; DB indexes via primary keys; cart stores unit-price snapshot to avoid extra price lookups at checkout
Scalability	Stateless API instances can be scaled on Render; SPA scaled via Vercel; database on managed Neon; modular packages allow future extraction of services if needed

Availability	Managed PaaS (Render/Vercel/Neon) handles OS/patching; free-tier cold starts are a known trade-off documented in the reflection
--------------	---

13 Web Design

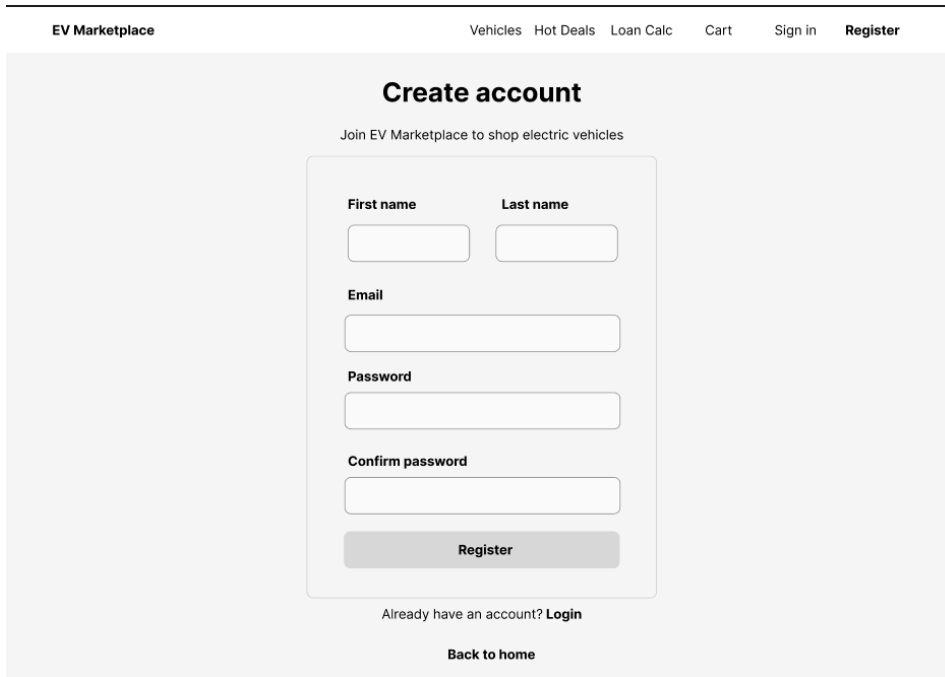
The EV Marketplace front end is a React single-page application styled with Tailwind CSS. The design aims for a clean, product-focused layout: light gray page backgrounds, white content cards, a consistent top navigation bar, and dark primary buttons with a blue hover state. Navigation is shared across pages (Vehicles, Hot Deals, Loan Calc, Cart, Sign in / Register) so users can move between catalog, deals, financing, and account flows without relearning the chrome. The screens below are taken from the production build at <https://evmarketplace.vercel.app> and illustrate the main customer-facing interfaces.

Sign in page:

Figure 8: Sign in page UI

Figure 8 shows the Sign in screen. A centered white card sits on a neutral background with the title “Sign in” and a short welcome line. The form has two fields, email and password, and a full-width primary button. Secondary links below the card route users to registration or back home. The layout is minimal on purpose: no competing sidebars or marketing clutter, so authentication feels quick and low-friction before browsing or checkout.

Register page:

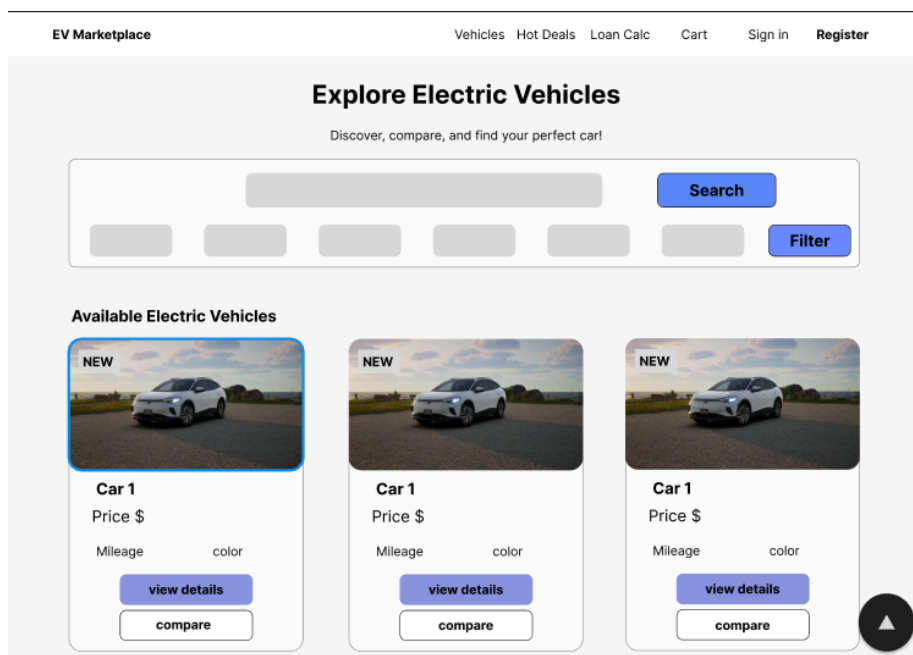


The register page features a navigation bar with links: EV Marketplace, Vehicles, Hot Deals, Loan Calc, Cart, Sign in, and Register. The main heading is 'Create account' with a subtext 'Join EV Marketplace to shop electric vehicles'. The form includes fields for First name, Last name, Email, Password, and Confirm password, followed by a Register button. Below the form is a link for existing users to Login and a Back to home link.

Figure 9: Register page UI

Figure 9 shows account creation. The same card-centered pattern as Sign in is reused for visual consistency. Fields collect first name, last name, email, password, and password confirmation, with a primary “Register” action and a link back to Sign in for existing users. Grouping name fields on one row and stacking credentials below keeps the form scannable on desktop and mobile. Validation feedback (e.g. mismatch or missing fields) is intended to appear inline without leaving the page.

Home page:



The home page features a navigation bar with links: EV Marketplace, Vehicles, Hot Deals, Loan Calc, Cart, Sign in, and Register. The main heading is 'Explore Electric Vehicles' with a subtext 'Discover, compare, and find your perfect car!'. Below the heading is a search bar with a Search button and a Filter button. The section 'Available Electric Vehicles' displays three car cards, each with a 'NEW' badge, a car image, the title 'Car 1', price, mileage, color, and buttons for 'view details' and 'compare'. A scroll indicator is visible at the bottom right.

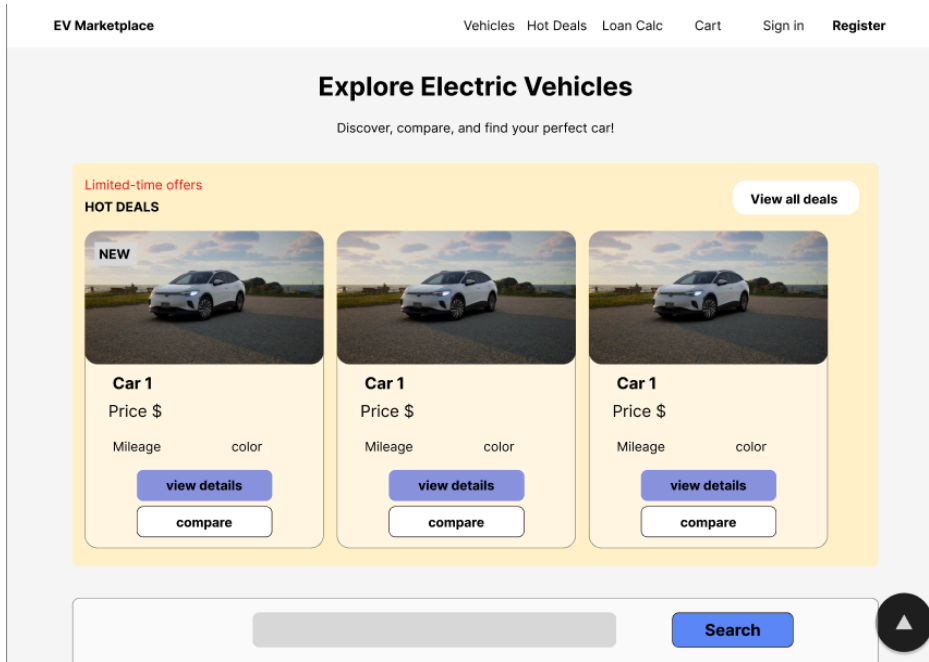


Figure 10: Home page UI

Figure 10 is the main catalog view. The navbar anchors the top; below it, a search and filter strip lets users narrow the vehicle list. Results appear as a responsive card grid: each card shows a vehicle image, condition badge (e.g. NEW), price, mileage/color cues, and actions such as “View details” and “Compare.” This card pattern is the primary browsing metaphor for the store, scan images and prices first, then open a detail page, rather than a dense table of specs.

Hot deals:

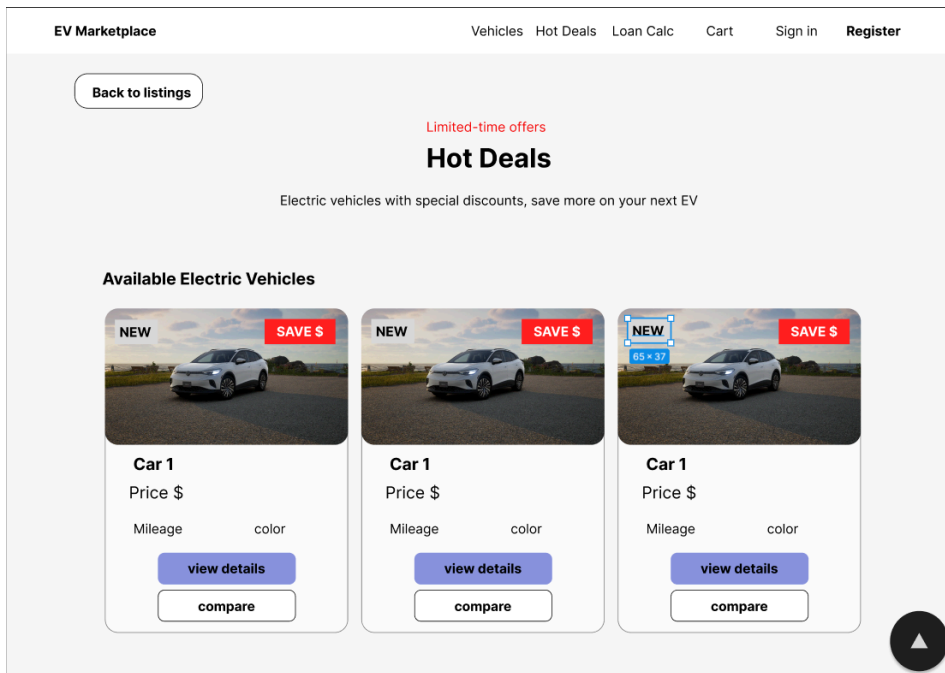


Figure 11: Hot deals page UI

Figure 11 presents promotional inventory. The page reuses the catalog card language but emphasizes discounted vehicles (e.g. strikethrough original price and a savings or badge

treatment). The goal is to surface value quickly for users who open “Hot Deals” from the navbar instead of filtering the full catalog. Layout stays consistent with Home so the only mental shift is “these items are on promotion,” not a different UI system.

Vehicle Details page:

EV Marketplace
Vehicles
Hot Deals
Loan Calc
Cart
Sign in
Register

Back to listings



☆☆☆☆ 4.0 (5 reviews)

used
Car 1
Price \$

Vehicle Specifications

MileageVINColorCondition

No reported accidents on record

▲

EV Marketplace
Vehicles
Hot Deals
Loan Calc
Cart
Sign in
Register

Customer Reviews

Write a review

Your nameRating

Comment

Submit

User1
7/20/2026
☆☆☆☆
Fantastic car, good for family we love it!

User1
7/20/2026
☆☆☆☆
Fantastic car, good for family we love it!

Customize this vehicle

▲

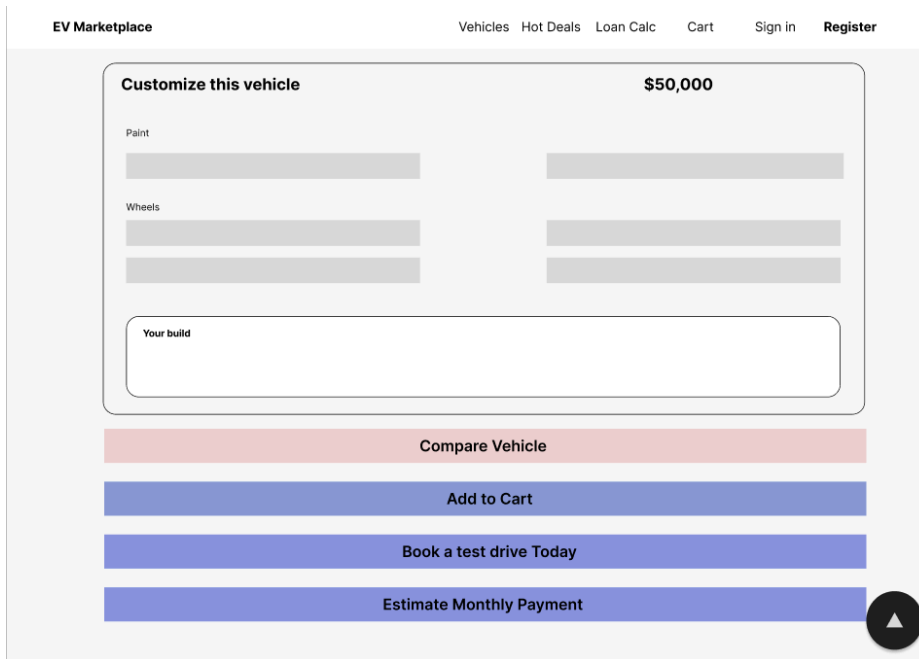


Figure 12: Vehicle details page UI

Figure 12 shows a single vehicle's detail view. A large image area (or 360-style viewer when multiple photos exist) sits above a summary of make, model, year, condition, and pricing, including original price, discount, and final price when a deal applies. Average star rating and review count appear near the title so social proof is visible before the user scrolls. Specifications (mileage, VIN, color, condition) are grouped in a clear grid. Below that, customer reviews list reviewer name, star rating, and comment, with a short form to submit a new review. Primary actions: Add to Cart, Book a Test Drive, and Estimate Monthly Payment, use full-width dark buttons consistent with the rest of the site so the next step is obvious without competing layouts.

Loan calculator:

EV Marketplace Vehicles Hot Deals Loan Calc Cart Sign in Register

[Back to listings](#)

Loan Calculator

Vehicle Price

Down Payment

Trade-in Value

Interest Rate (APR)

Loan Term

Estimated Monthly Payment

\$600/mo

Loan amount: \$10,000

Total Interest: \$5,000

[Back to listings](#)

Figure 13: Loan calculator page UI

Figure 13 is the financing estimator. Inputs typically include vehicle price (optionally prefilled from a selected vehicle), down payment, interest rate, and term; the UI then shows an estimated monthly payment. The page is intentionally a standalone tool rather than part of checkout, matching the product decision that loan estimates help decision-making before purchase. Clear labels and a single result area keep the calculator readable for users who are not finance experts.

Cart:

EV Marketplace Vehicles Hot Deals Loan Calc Cart Sign in Register

[Continue Shopping](#)

Shopping Cart

Signed in as user@name

Car 1

Color x \$30,000 [Remove](#)

Sub-total	\$20,000
HST	\$1,000
Total	\$21,000

[Proceed to Checkout](#)

[Back to listings](#)

Figure 14: Cart page UI

Figure 14 shows the shopping cart. Line items are listed with vehicle identity, unit price (snapshot from when the item was added), quantity controls, and a remove action. A running total is shown so the user can review the order before checkout. The page reuses the same navbar and card/table styling as the catalog so cart management feels like a continuation of browsing rather than a separate site. Primary navigation continues to “Checkout” once the user is satisfied with quantities.

Checkout:

EV Marketplace Vehicles Hot Deals Loan Calc Cart Sign in Register

[Back to Cart](#)

Checkout

Signed in as user@name

Billing information

Billing name

Billing address

Shipping Information ☐ Same as billing

Card Number

Expiry Date CVV

Order Summary

Sub-total	\$20,000
HST	\$1,000
Total	\$21,000

[Confirm Order](#)

Figure 15: Checkout page UI

Figure 15 is the checkout confirmation step. It summarizes the cart (or resulting order) and triggers the server-side order creation and simulated payment flow. Success and failure are both surfaced in the UI, for example a confirmed order with total, or a clear “Credit Card Authorization Failed” style message when payment is denied, so the outcome of checkout is never ambiguous. Layout stays minimal: focus on order summary and status rather than extra marketing content.

Book a test Drive:

EV Marketplace Vehicles Hot Deals Loan Calc Cart Sign in Register

Back

Book a Test Drive

Vehicle ID: 1

Customer Information

Signed in as user@name

[view my bookings](#)

Pick a time

< Sep >

2025

>

Su	Mo	Tu	We	Th	Fr	Sa
	1	2	3	4	5	6
7	8	9	10	11	12	13
14	15	16	17	18	19	20
21	22	23	24	25	26	27
28	29	30	1	2	3	4

Confirm Test Drive

Figure 16: Test Drive page

Figure 16 shows the Book a Test Drive page, where users can schedule a test drive by selecting an available date from the integrated calendar. Test drives are booked through the user's account, and once a booking is confirmed, it is automatically displayed in the user's booking history for future reference. The interface features a clean, intuitive, and minimalistic layout, making it easy for users to navigate and complete the booking process. This design aligns with user experience principles by providing an accessible, user-friendly, and straightforward interface that enhances usability for all users.

14 Activities Plan for Deliverable 3

Backlog ID	Item	Status	Deliverable
PB-20	Chatbot (backend + frontend widget)	Complete — hybrid intents + catalog queries; quick-reply chips after demo feedback	Deliverable 3
PB-23	Curl / Postman test collection	Complete — With Section 10	Deliverable 3
PB-24	SQL injection + XSS security tests	Complete — TC-SEC-01, TC-SEC-02; shown in demo	Deliverable 3
PB-25	Performance / load test (JMeter)	Complete — TC-PERF-01; .jmx in repo	Deliverable 3

PB-26	Cloud deploy (HTTPS)	Complete — API on Render, SPA on Vercel; Neon DB; pivoted from AWS EB / Learner Lab limits	Deliverable 3
PB-28	Home / catalog / vehicle detail UI	Complete	Deliverable 3
PB-29	Cart + checkout UI	Complete	Deliverable 3
PB-30	Login / register / sign-out UI	Complete	Deliverable 3
PB-31	Reviews + 5-star ratings UI	Complete	Deliverable 3
PB-32	Hot deals view	Complete	Deliverable 3
PB-33	Loan calculator UI	Complete	Deliverable 3
PB-34	Test-drive booking UI	Complete	Deliverable 3
PB-35	Figma / UI screenshots for web-design section	Complete	Deliverable 3
PB-27	Complete Deliverable 3 documentation	Complete	Deliverable 3

15 Client-Side Implementation Description

15.1 Cart/Checkout/Payment

Technologies used

- React (function components with hooks) bootstrapped via Vite, consistent with the rest of the frontend.
- react-router-dom for client-side routing: /cart and /checkout endpoints are registered as top-level routes in App.jsx alongside Home, following the same convention established for the catalogue pages.
- Tailwind CSS utility classes for layout and styling.
- Native Fetch API (no axios or other HTTP client) wrapped in two thin service modules—CartService.jsx and OrderService.jsx—matching the pattern already used in VehicleService.jsx.

Component / page breakdown

Cart.jsx

- Loads the current user's cart contents via `CartService.getCart(userId)` and renders each `CartItem` with its server-resolved unit price, quantity, and line subtotal.
- Computes and displays the running cart total client-side from the returned line items (the authoritative total is still whatever the server would compute at checkout; no total is trusted from client state alone).
- Provides a remove action per line item, calling `CartService.removeCartItem(cartItemId)`, and a link through to `Checkout.jsx`.

Checkout.jsx

- Calls `OrderService.checkout(userId)`, which triggers the server-side order-creation and fake-payment-approval flow described in Section 8.5.
- Renders the resulting order status, distinguishing a `CONFIRMED` order (with its item and total) from a `DENIED` payment attempt.
- Reproduces the required user-facing 'Credit Card Authorization Failed' message on a `DENIED` result, matching test case TC-11 from the test suite.

CartService.jsx / OrderService.jsx

- Pure data-access modules: each exported function performs one fetch call, checks `response.ok`, and either returns parsed JSON or throws – no business logic (pricing, payment rules) lives on the client, consistent with the server-side design described in Section 8.5.
- `getCart`, `addToCart`, `removeCartItem`, and `clearCart` cover the full Shopping Cart REST surface; `checkout`, `getOrder`, and `getOrdersByUser` cover the Ordering REST surface.

REST principles followed (client side)

- **Resource-oriented calls:** the client always issues `GET/POST/DELETE` against `/api/cart` and `/api/orders` resources rather than RPC-style endpoints, mirroring the server-side design.
- **Statelessness:** no cart or order data is cached in client-side storage between page loads, every page re-fetches from the server on mount, so the UI always reflects authoritative server state.
- **Separation of concerns:** page components (`Cart.jsx`, `Checkout.jsx`) contain only rendering and user-interaction logic; all HTTPS calls are isolated in the service modules.

15.2 Test Drive Booking

Technologies used

The Test Drive feature was developed using React function components with Hooks, `react-router-dom` for the `/test-drive/:vehicleId` route, `react-calendar` for date selection, Tailwind CSS for responsive styling, and a `TestDriveService.jsx` fetch wrapper following the same architecture as `CartService.jsx` and `VehicleService.jsx`.

Component breakdown:

- `TestDrive.jsx` integrates the `react-calendar` component to allow users to select a preferred test drive date. When a date is chosen, the application retrieves and displays

the available time slots for that specific vehicle through `TestDriveService.getAvailableSlots()`. Users can then select one of the available time slots and confirm their booking. The component also displays the user's existing bookings and provides an option to cancel them.

- Slot selection is restricted to server-generated candidates only; the client never constructs an arbitrary date or time. This ensures that every booking request is validated against the backend's availability and conflict-checking logic.
- If two users attempt to book the same time slot simultaneously, the backend detects the conflict using `SERIALIZABLE` transaction isolation and returns a 409 Conflict response. The client displays an appropriate message and automatically refreshes the available time slots.
- `TestDriveService.jsx` functions as a dedicated data-access layer responsible for communicating with the backend API. It contains no booking or conflict-resolution logic, maintaining a clear separation of concerns and adhering to the statelessness principle described in Section 8.

REST principles followed: The implementation follows RESTful design principles through resource-oriented API calls to `/api/test-drives`. Booking state is maintained by the backend, with no client-side caching between page loads, ensuring that users always view the most up-to-date availability and booking information.

16 Chatbot Implementation

The chatbot is implemented as a hybrid system combining rule-based intent matching with catalog-grounded responses, rather than either static canned text or a full external LLM integration. This keeps the feature self-contained, free to run, and directly verifiable against the live database.

- **Backend (`ChatbotService`, `ChatbotController`):** incoming messages are lower-cased and checked against keyword sets in priority order (greeting, hot deals, vehicle search, test drive help, loan calculator help, checkout help, reviews, support fallback). Two intents, hot deals and vehicle search, are catalog-grounded: instead of returning fixed text, the service queries `VehicleRepository` live, parsing make names (matched dynamically against the actual seeded data rather than a hardcoded list) and price ceilings (e.g. "under \$50000" via regex) directly from the user's message, and returns real matching vehicles alongside the reply text.
- **Frontend (`ChatWidget.jsx`):** following feedback from the D3 demo, where a free-text question outside the keyword set returned the fallback response, quick-reply suggestion buttons were added alongside the free-text input. These buttons send a pre-defined phrase that is guaranteed to match a known intent (e.g. "Show me hot deals," "How do I book a test drive?"), so a user can always get an accurate response without needing to guess phrasing, while free-text entry remains available for more specific queries like vehicle search.

EV Marketplace Assistant



Hi! Ask me about vehicles, hot deals, test drives, loans, or checkout. You can also tap a suggestion below.

Show hot deals

Find Tesla under 60000

How do I book a test drive?

Help with checkout

How does the loan calculator work?

How do I leave a review?

Type a message...

Send

17 Demo Table

Feature to test/check during the demo (35% of Deliverables 3)	Mark	Comment made by the TA/prof
1) The system is cloud-native (4%)		
2) It is possible to register (2%)		
3) It is possible to sign in (2%)		
4) It is possible to sign out (2%)		
5) The system is secure (e.g., the system is using https in its url) (3%)		
6) It is possible to filter items (2%)		
7) It is possible to sort items (2%)		
8) It is possible to add items in the shopping cart (2%)		
9) It is possible to remove items from the shopping cart (2%)		
10) It is possible to check out items (2%)		
11) It is possible to write reviews on items and rate items using five stars (2%)		

12) It is possible to use the distinguished feature (2%)		
13) It is possible to use the chatbot (3%)		
14) It is possible to use the loan calculator (3%)		
15) It is possible to view hot deals (2%)		

18 Reflection

- Strengths:** The codebase is structured as a modular monolith with clear packages (catalog, cart, ordering, identity, test drive, analytics, notification, chatbot). That kept modules independently testable and allowed parallel work with limited merge conflicts. The distinguished feature, test-drive booking, uses serializable transaction isolation to reduce double-booking under concurrent requests, which we verified in testing. The chatbot combines rule-based intent matching with live catalog queries rather than only static replies, and was improved after demo feedback with quick-reply suggestion chips so common questions map to accurate intents. When our initial AWS Elastic Beanstalk deployment hit AWS Academy Learner Lab IAM restrictions that blocked HTTPS entirely, we migrated the whole deployment to Render and Vercel rather than shipping without HTTPS, resulting in a more resilient and permanently hosted deployment than originally planned. We also identified a real security gap during testing; passwords were being stored in plaintext and remediated it before submission by migrating to BCrypt hashing with a backward-compatible upgrade path so already-registered test accounts continue to work. Load testing with JMeter showed zero request failures across all concurrency levels tested, up to 50 concurrent users.
- Weaknesses:** Session/token handling and role-based access checks on admin reporting endpoints are not yet production-hardened; with more time we would add proper authorization guards so only admin-role users can access the reports endpoints, and move session tokens from in-memory storage to a persisted, revocable store. Deployment consumed disproportionate effort: an initial AWS Elastic Beanstalk path under an AWS Academy Learner Lab hit IAM limits (including S3/CloudFront-related restrictions), which blocked a simple HTTPS front door on the default Beanstalk hostname and forced a late pivot to **Render** (API) and **Vercel** (SPA) with Neon PostgreSQL. Some scope boundaries were deliberate rather than oversights: the Loan Calculator is a standalone estimation tool, not integrated into the actual checkout payment flow; services are logically separated by responsibility in the codebase but run as a single deployed Spring Boot application rather than physically separate microservice containers, given the project timeline; and vehicle body-type and accident-history data is seeded manually rather than sourced from a real third-party vehicle history provider.
- Enjoyed most:** Building test-drive booking end-to-end, concurrency-aware backend plus a usable booking UI, and seeing conflicting bookings rejected correctly in tests. Be able to implement simple 360 images for the Details car images.
- Enjoyed least:** Cloud account and TLS constraints on the learner environment, which delayed a stable public HTTPS demo URL relative to feature work.
- Technology we would have liked in class:** A basic **CI/CD** pipeline (e.g. GitHub Actions) to run tests and deploy on merge would have reduced manual deploy risk near the demo.

- **Other comments:** The phased deliverables (design → server → client/deployment) spread the load well, but early infrastructure choices sometimes had to be revisited when real platform limits appeared. Overall the stack (React, Spring Boot, PostgreSQL, Vercel, Render) was a practical fit for a cloud-native course project.